

```

/*-----*/
#include <medusa/view/GraphicView.h>
/*-----*/
#include <vtkRenderer.h>
#include <vtkRenderWindow.h>
#include <vtkRenderWindowInteractor.h>
#include <vtkProperty.h>
#include <vtkDataSetMapper.h>
#include <vtkCamera.h>
#include <vtkCellPicker.h>
#include <vtkCellData.h>
/*-----*/
#include <string>
#include <gmds/utils/Exception.h>
#include <medusa/model/MedusaBackEnd.h>
#include <medusa/control/MouseInteractorStyle.h>
#include <vtkThreshold.h>
#include <vtkScalarBarActor.h>
#include <vtkLookupTable.h>
#include <vtkLine.h>
#include <vtkSphereSource.h>
/*-----*/
using namespace medusa;
/*-----*/
GraphicView::GraphicView(const std::string& AName, GraphicView::ViewType AType)
    : View(), m_name(AName), m_type(AType)
{
    m_visible=false;

    //We initialize a picking listener ont this view
    m_vtk_renderer = vtkRenderer::New();
    m_vtk_renderer->SetBackground( 0.92, 0.92, 0.92 );

    m_vtk_window = vtkRenderWindow::New();
    m_vtk_window->SetSize( 1000, 800 );
    m_vtk_window->AddRenderer( m_vtk_renderer );
    m_vtk_window->SetWindowName(m_name.c_str());
    m_vtk_interactor= vtkRenderWindowInteractor::New();
    m_vtk_interactor->SetRenderWindow(m_vtk_window);
    //Initialization is mandatory to be displayed
    m_vtk_interactor->Initialize();

    // Set the custom stype to use for interaction.
    m_mouse=
        vtkSmartPointer<MouseInteractorStyle>::New();
    m_mouse->SetDefaultRenderer(m_vtk_renderer);
    m_vtk_interactor->SetInteractorStyle(m_mouse);

    selectedMapper = vtkSmartPointer<vtkDataSetMapper>::New();
    selectedActor = vtkSmartPointer<vtkActor>::New();

    m_vtk_cut_actor = vtkSmartPointer<vtkActor>::New();

    m_vtk_tets_set = vtkSmartPointer<vtkActor>::New();
    m_vtk_renderer->AddActor(m_vtk_tets_set);

    m_mapper_dual = vtkDataSetMapper::New();
    m_mapper_dual->CreateDefaultLookupTable();

    m_mapper_dual->GetLookupTable()->Build();

    m_textActor = vtkSmartPointer<vtkTextActor>::New();
    m_textActor->SetPosition2 ( 10, 40 );
    m_textActor->GetTextProperty()->SetFontSize ( 24 );
    m_textActor->GetTextProperty()->SetColor ( 1.0, 0.0, 0.0 );
    m_vtk_renderer->AddActor2D ( m_textActor );
}
/*-----*/
GraphicView::~GraphicView() {
    m_vtk_renderer->Delete();
    m_vtk_window->Delete();
    m_vtk_interactor->Delete();
    for(auto a: m_vtk_actors)
        a->Delete();
}
/*-----*/
void GraphicView::setVisibleOFF() {m_visible=false;}
/*-----*/
void GraphicView::setVisibleON() {m_visible=true;}
/*-----*/
void GraphicView::refresh() {
    setVisibleON();
    //Render the scene .
    m_vtk_window->Render();
}
/*-----*/
void GraphicView::selectVTKCell(int ADIm, vtkIdType ACellID) {
    vtkSmartPointer<vtkNamedColors> colors =
        vtkSmartPointer<vtkNamedColors>::New();

```

```

vtkSmartPointer<vtkIdTypeArray> ids =
    vtkSmartPointer<vtkIdTypeArray>::New();
ids->SetNumberOfComponents(1);
ids->InsertNextValue(ACellID);

vtkSmartPointer<vtkSelectionNode> selectionNode =
    vtkSmartPointer<vtkSelectionNode>::New();
selectionNode->SetFieldType(vtkSelectionNode::CELL);
selectionNode->SetContentType(vtkSelectionNode::INDICES);
selectionNode->SetSelectionList(ids);

vtkSmartPointer<vtkSelection> selection =
    vtkSmartPointer<vtkSelection>::New();
selection->AddNode(selectionNode);

vtkSmartPointer<vtkExtractSelection> extractSelection =
    vtkSmartPointer<vtkExtractSelection>::New();
extractSelection->SetInputData(0, MedusaBackend::getInstance()->grids()[0]->grid());
extractSelection->SetInputData(1, selection);
extractSelection->Update();

// In selection
vtkSmartPointer<vtkUnstructuredGrid> selected =
    vtkSmartPointer<vtkUnstructuredGrid>::New();
selected->ShallowCopy(extractSelection->GetOutput());

selectedMapper->SetInputData(selected);
selectedActor->SetMapper(selectedMapper);
selectedActor->GetProperty()->EdgeVisibilityOn();
selectedActor->GetProperty()->SetColor(
    colors->GetColor3d("Red").GetData());

selectedActor->GetProperty()->SetLineWidth(5);
selectedActor->GetProperty()->SetPointSize(5);

m_vtk_renderer->AddActor(selectedActor);
m_vtk_window->Render();
}

/*-----*/
void GraphicView::update() {
    //means the backend changed, so we update our local pipeline to produce the
    //expected actors

    //clean current actors
    for(auto a: m_vtk_actors) {
        m_vtk_renderer->RemoveActor(a);
        a->Delete();
    }
    if(m_vtk_actors.size() > 2) {
        m_vtk_actors.erase(m_vtk_actors.begin());
        std::cout << "test1" << std::endl;
        m_vtk_actors.erase(m_vtk_actors.begin());
        std::cout << "test2" << std::endl;
    }

    m_vtk_cut_actor->VisibilityOff();
    m_vtk_cut_actor->PickableOff();

    std::cout<<"nb actors left "<<m_vtk_actors.size()<<std::endl;

    m_vtk_actors.clear();
    std::vector<MedusaGrid*> grids = MedusaBackend::getInstance()->grids();
    m_vtk_actors.resize(grids.size());
    int i =0;
    for(auto g:grids) {
        std::cout<<"View: grid name = "<<g->name<<std::endl;
        vtkDataSetMapper *mapper = vtkDataSetMapper::New();
        mapper->SetInputData(g->grid());
        vtkSmartPointer<vtkActor> actor = vtkActor::New();
        actor->SetMapper(mapper);
        actor->GetProperty()->SetDiffuseColor(0.5,0.5,0.5);
        actor->VisibilityOff();
        m_vtk_actors[i++]=actor;
        m_vtk_renderer->AddActor(actor);
    }

    m_vtk_actors[0]->VisibilityOn();
    m_vtk_actors[0]->GetProperty()->EdgeVisibilityOn();
    m_vtk_actors[0]->GetProperty()->SetLineWidth(0.15);

    if(test == 1){
        m_vtk_renderer->RemoveActor(m_vtk_cut_actor);
        //m_vtk_cut_actor->Delete();
    }
    test = 1;

    if(m_sinGraph_visibility == -1) {
        // buildSingGraph(MedusaBackend::getInstance()->getSingGraph());
    }

```

```

    }
    textMode(0);
    m_vtk_renderer->Render();
}
/*-----*/
void GraphicView::updateMesh() {
    std::vector<MedusaGrid*> grids = MedusaBackend::getInstance()->grids();
    vtkDataSetMapper *mapper = vtkDataSetMapper::New();
    mapper->SetInputData(grids[0]->grid());
    m_vtk_actors[0]->SetMapper(mapper);
}
/*-----*/
void GraphicView::start() {m_vtk_interactor->Start();}
/*-----*/
void GraphicView::createSurface(const std::vector<vtkIdType> AIDs, int ASheetID) {

    vtkSmartPointer<vtkNamedColors> colors =
        vtkSmartPointer<vtkNamedColors>::New();

    vtkSmartPointer<vtkIdTypeArray> ids =
        vtkSmartPointer<vtkIdTypeArray>::New();
    ids->SetNumberOfComponents(1);
    for(auto id:AIDs)
        ids->InsertNextValue(id);

    vtkSmartPointer<vtkSelectionNode> selectionNode =
        vtkSmartPointer<vtkSelectionNode>::New();
    selectionNode->SetFieldType(vtkSelectionNode::CELL);
    selectionNode->SetContentType(vtkSelectionNode::INDICES);
    selectionNode->SetSelectionList(ids);

    vtkSmartPointer<vtkSelection> selection =
        vtkSmartPointer<vtkSelection>::New();
    selection->AddNode(selectionNode);

    vtkSmartPointer<vtkExtractSelection> extractSelection =
        vtkSmartPointer<vtkExtractSelection>::New();
    extractSelection->SetInputData(0, MedusaBackend::getInstance()->grids()[0]->grid());
    extractSelection->SetInputData(1, selection);
    extractSelection->Update();

    // In selection
    vtkSmartPointer<vtkUnstructuredGrid> selected =
        vtkSmartPointer<vtkUnstructuredGrid>::New();
    selected->ShallowCopy(extractSelection->GetOutput());

    std::vector<double> color = {(double)((rand()%256))/256, (double)((rand()%256))/256, (double)((rand()%256))/256};
    m_surf_to_colors.emplace(ASheetID, color);

    vtkDataSetMapper *mapper_selection = vtkDataSetMapper::New();
    mapper_selection->SetInputData(selected);
    mapper_selection->ScalarVisibilityOff();
    vtkSmartPointer<vtkActor> actor_selection = vtkActor::New();
    actor_selection->SetMapper(mapper_selection);
    actor_selection->GetProperty()->EdgeVisibilityOff();
    actor_selection->GetProperty()->SetLineWidth(0.25);
    actor_selection->GetProperty()->SetColor(color[0],color[1],color[2]);

    if(surface_mode == 1){
        actor_selection->VisibilityOff();
    }
    //setOpacityOFF();
    m_vtk_tets_actors.insert(std::pair<vtkSmartPointer<vtkActor>,int>(actor_selection,ASheetID));

    actor_selection->GetProperty()->SetLineWidth(5);
    actor_selection->GetProperty()->SetPointSize(5);
    actor_selection->PickableOff();

    m_vtk_renderer->AddActor(actor_selection);

    m_vtk_window->Render();
}
/*-----*/
void GraphicView::updateDual(int ANbZones) {

    m_nb_zones = ANbZones;

    vtkDataArray *array = MedusaBackend::getInstance()->grids()[0]->grid()->GetCellData()->GetArray("blocks");
    MedusaBackend::getInstance()->grids()[0]->grid()->GetCellData()->SetScalars(array);

    vtkSmartPointer<vtkThreshold> threshold_zone = vtkThreshold::New();
    threshold_zone->SetInputData(MedusaBackend::getInstance()->grids()[0]->grid());
    if(ANbZones == -2){
        threshold_zone->ThresholdByLower(-2);
    }else{
        threshold_zone->ThresholdByUpper(1);
    }
}

```

```

threshold_zone->SetInputArrayToProcess(0, 0, 0, vtkDataObject::FIELD_ASSOCIATION_CELLS, "blocks");
threshold_zone->Update();
vtkUnstructuredGrid* thresholdedPolydata = threshold_zone->GetOutput();

m_mapper_dual->SetInputData(thresholdedPolydata);
m_mapper_dual->SetScalarModeToUseCellData();
m_mapper_dual->SetColorModeToMapScalars();

m_mapper_dual->ScalarVisibilityOn();
m_mapper_dual->SetScalarRange(-2,10);
m_mapper_dual->Update();

m_vtk_actors[0]->SetMapper(m_mapper_dual);

for(auto a : m_vtk_tets_actors){
    a.first->VisibilityOff();
}

m_vtk_renderer->Render();
}
/*-----*/
void GraphicView::updateCut() {

    vtkDataArray *array = MedusaBackend::getInstance()->grids()[0]->grid()->GetCellData()->GetArray("cut");

    MedusaBackend::getInstance()->grids()[0]->grid()->GetCellData()->SetScalars(array);

    vtkSmartPointer<vtkThreshold> threshold_zone = vtkThreshold::New();
    threshold_zone->AddInputData(MedusaBackend::getInstance()->grids()[0]->grid());
    threshold_zone->ThresholdByLower(1);
    threshold_zone->SetInputArrayToProcess(0, 0, 0, vtkDataObject::FIELD_ASSOCIATION_CELLS, "cut");
    threshold_zone->Update();
    vtkUnstructuredGrid* thresholdedPolydata = threshold_zone->GetOutput();

    vtkDataSetMapper *m_mapper_cut = vtkDataSetMapper::New();
    m_mapper_cut->SetInputData(thresholdedPolydata);
    m_mapper_cut->SetScalarModeToUseCellData();
    m_mapper_cut->SetColorModeToDefault();
    m_mapper_cut->ScalarVisibilityOff();
    m_mapper_cut->SetScalarRange(0,1);

    m_vtk_actors[0]->SetMapper(m_mapper_cut);
    //m_vtk_actors[0]->GetProperty()->SetColor(0,0,0);

    removeSelection();

    //En fait c'est l'acteur pour les parties coupées
    vtkSmartPointer<vtkThreshold> threshold_cut = vtkThreshold::New();
    threshold_cut->AddInputData(MedusaBackend::getInstance()->grids()[0]->grid());
    threshold_cut->ThresholdByUpper(2);
    threshold_cut->SetInputArrayToProcess(0, 0, 0, vtkDataObject::FIELD_ASSOCIATION_CELLS, "cut");
    threshold_cut->Update();
    vtkUnstructuredGrid* thresholdedPolydata_cut = threshold_cut->GetOutput();

    vtkDataSetMapper *mapper = vtkDataSetMapper::New();
    mapper->SetInputData(thresholdedPolydata_cut);
    mapper->SetScalarModeToUseCellData();
    mapper->SetColorModeToMapScalars();
    mapper->ScalarVisibilityOn();
    mapper->SetScalarRange(1,2);

    m_vtk_cut_actor->SetMapper(mapper);
    //Attention on a changé l'opacité ici
    m_vtk_cut_actor->VisibilityOff();
    m_vtk_cut_actor->GetProperty()->SetOpacity(0.5);

    m_vtk_renderer->AddActor(m_vtk_cut_actor);

    m_vtk_window->Render();
}
/*-----*/
void GraphicView::setDualVisibilityON() {
    for(auto a:m_vtk_tets_actors){
        a.first->VisibilityOn();
        a.first->GetProperty()->SetDiffuseColor(0.5,0.5,0.5);
        //a->GetProperty()->SetColor(1.0,0.0,0.0);
    }
    for(auto a:m_vtk_surface_actors){
        a.first->VisibilityOn();
        a.first->GetProperty()->SetDiffuseColor(0.5,0.5,0.5);
        //a->GetProperty()->SetColor(1.0,0.0,0.0);
    }
    MedusaGrid* grid = MedusaBackend::getInstance()->grids()[0];
    vtkDataSetMapper *mapper = vtkDataSetMapper::New();
    mapper->SetInputData(grid->grid());
    mapper->SelectColorArray("blocks");
    mapper->SetColorModeToMapScalars();

```

```

    mapper->SetScalarModeToUseCellFieldData();
    mapper->ScalarVisibilityOn();
    mapper->SetScalarRange(-1,m_nb_zones-1);
    mapper->Update();

    vtkSmartPointer<vtkActor> actor = vtkActor::New();
    m_vtk_actors[0]->SetMapper(mapper);
    m_vtk_actors[0]->GetProperty()->SetOpacity(1.0);
}
/*-----*/
void GraphicView::setDualVisibilityOFF() {
    for(auto a:m_vtk_tets_actors){
        a.first->VisibilityOff();
        //a->GetProperty()->SetColor(1.0,0.0,0.0);
    }
    for(auto a:m_vtk_surface_actors){
        a.first->VisibilityOff();
        //a->GetProperty()->SetColor(1.0,0.0,0.0);
    }

    m_vtk_actors[0]->SetMapper(m_mapper_dual);

    m_vtk_actors[0]->VisibilityOn();
    m_vtk_actors[0]->GetProperty()->SetRepresentationToSurface();
}
/*-----*/
void GraphicView::wireframe() {
    if (m_mode == 0) {
        for (auto a:m_vtk_actors) {
            a->GetProperty()->SetRepresentationToSurface();
        }
        m_vtk_actors[0]->GetProperty()->SetRepresentationToWireframe();
    }else{
        for (auto a:m_vtk_actors) {
            a->GetProperty()->SetRepresentationToWireframe();
        }
        m_vtk_actors[0]->GetProperty()->SetRepresentationToSurface();
    }
    m_vtk_window->Render();
}
/*-----*/
void GraphicView::wireframeAll() {
    for(auto a:m_vtk_actors){
        a->GetProperty()->SetRepresentationToWireframe();
    }
    m_vtk_window->Render();
}
/*-----*/
void GraphicView::solid() {
    for (auto a:m_vtk_actors) {
        a->GetProperty()->SetRepresentationToSurface();
    }
    m_vtk_window->Render();
}
/*-----*/
void GraphicView::opacity() {
    if(m_opacity){
        setOpacityOFF();
    }else{
        setOpacityON();
    }

    m_vtk_window->Render();
}
/*-----*/
void GraphicView::setOpacityON() {
    if(m_mode == 0){
        m_vtk_actors[0]->GetProperty()->SetOpacity(1.0);
    }else{
        setDualVisibilityOFF();
    }
    m_opacity = true;
}
/*-----*/
void GraphicView::setOpacityOFF() {
    if(m_mode == 0){
        m_vtk_actors[0]->GetProperty()->SetOpacity(0.25);
    }else{
        setDualVisibilityON();
    }
    m_opacity = false;
}
/*-----*/
void GraphicView::toggleCutOpacity() {
    if(m_op_cut == 2){
        m_vtk_cut_actor->GetProperty()->SetOpacity(0);
        m_op_cut = 0;
    }else if(m_op_cut == 0){

```

```

        m_vtk_cut_actor->GetProperty()->SetOpacity(0.5);
        m_op_cut = 1;
    }else{
        m_vtk_cut_actor->GetProperty()->SetOpacity(1);
        m_op_cut = 2;
    }
}
/*-----*/
// Maybe not necessary
void GraphicView::unlockSheet(){
    if(surface_mode == 0) {
        for (auto a:m_vtk_tets_actors) {
            a.first->PickableOn();
        }
    }else{
        for (auto a:m_vtk_surface_actors) {
            a.first->PickableOn();
        }
    }
    m_vtk_actors[0]->PickableOff();
}
/*-----*/
// Maybe not necessary
void GraphicView::unlockLines(){

    for (auto a:m_vtk_sing_actors) {
        std::cout<<a.first<<std::endl;
        a.first->PickableOn();
    }

    m_vtk_actors[0]->PickableOff();
}
/*-----*/
int GraphicView::selectLine(const vtkSmartPointer<vtkActor> ADeletedActor){

    ADeletedActor->GetProperty()->SetDiffuseColor(0,0,1);
    for(auto const &line : m_vtk_sing_actors){
        if(ADeletedActor == line.first){
            return line.second;
        }
    }
}
/*-----*/
void GraphicView::showSetOfTets(std::vector<vtkIdType> AIDs){

    vtkSmartPointer<vtkIdTypeArray> ids =
        vtkSmartPointer<vtkIdTypeArray>::New();
    ids->SetNumberOfComponents(1);
    for(auto id:AIDs)
        ids->InsertNextValue(id);

    vtkSmartPointer<vtkSelectionNode> selectionNode =
        vtkSmartPointer<vtkSelectionNode>::New();
    selectionNode->SetFieldType(vtkSelectionNode::CELL);
    selectionNode->SetContentType(vtkSelectionNode::INDICES);
    selectionNode->SetSelectionList(ids);

    vtkSmartPointer<vtkSelection> selection =
        vtkSmartPointer<vtkSelection>::New();
    selection->AddNode(selectionNode);

    vtkSmartPointer<vtkExtractSelection> extractSelection =
        vtkSmartPointer<vtkExtractSelection>::New();
    extractSelection->SetInputData(0, MedusaBackend::getInstance()->grids()[0]->grid());
    extractSelection->SetInputData(1, selection);
    extractSelection->Update();

    // In selection
    vtkSmartPointer<vtkUnstructuredGrid> selected =
        vtkSmartPointer<vtkUnstructuredGrid>::New();
    selected->ShallowCopy(extractSelection->GetOutput());

    vtkSmartPointer<vtkDataSetMapper> mapper = vtkDataSetMapper::New();
    mapper->SetInputData(selected);
    mapper->ScalarVisibilityOff();

    m_vtk_tets_set->SetMapper(mapper);
    m_vtk_tets_set->VisibilityOn();
    m_vtk_tets_set->GetProperty()->EdgeVisibilityOff();
    m_vtk_tets_set->GetProperty()->SetColor(1,0,0);
    m_vtk_tets_set->GetProperty()->SetLineWidth(5);
    m_vtk_tets_set->GetProperty()->SetPointSize(5);
    m_vtk_tets_set->PickableOff();

    m_vtk_window->Render();
}
/*-----*/
void GraphicView::addTetSetActor(){

```

```

    vtkSmartPointer<vtkActor> actor = vtkSmartPointer<vtkActor>::New();
    actor->SetMapper(m_vtk_tets_set->GetMapper());
    m_vtk_sets_actors.emplace_back(actor);
}
/*-----*/
void GraphicView::removeLastSelection() {
    auto actor = m_vtk_selection_actors.back();
    m_vtk_selection_actors.pop_back();
    m_vtk_renderer->RemoveActor(actor);
    actor->Delete();
}
/*-----*/
void GraphicView::removeSelection() {
    for(int i = 0; i<m_vtk_selection_actors.size();i++){
        m_vtk_renderer->RemoveActor(m_vtk_selection_actors[i]);
        m_vtk_selection_actors[i]->Delete();
    }
}
/*-----*/
int GraphicView::removeActor(const vtkSmartPointer<vtkActor> ADeletedActor) {

    int sheet_id = 0;
    if(surface_mode == 0) {
        for (auto a:m_vtk_tets_actors) {
            a.first->PickableOff();
            if (a.first == ADeletedActor) {
                sheet_id = a.second;
                m_vtk_renderer->RemoveActor(a.first);
                a.first->Delete();
            }
        }
        for(auto a:m_vtk_surface_actors){
            if(a.second == sheet_id){
                m_vtk_renderer->RemoveActor(a.first);
                a.first->Delete();
            }
        }
    } else{
        for (auto a:m_vtk_surface_actors) {
            a.first->PickableOff();
            if (a.first == ADeletedActor) {
                sheet_id = a.second;
                m_vtk_renderer->RemoveActor(a.first);
                a.first->Delete();
            }
        }
        for(auto a:m_vtk_tets_actors){
            if(a.second == sheet_id){
                m_vtk_renderer->RemoveActor(a.first);
                a.first->Delete();
            }
        }
    }
    m_surf_to_colors.erase(sheet_id);
    m_vtk_window->Render();
    m_vtk_actors[0]->PickableOn();
    return sheet_id;
}
/*-----*/
void GraphicView::removeActor(int ADeletedID) {

    for(auto a:m_vtk_tets_actors){
        if(a.second == ADeletedID){
            vtkSmartPointer<vtkActor> actor = a.first;
            m_vtk_renderer->RemoveActor(actor);
            a.first->Delete();
        }
    }
    for(auto a:m_vtk_surface_actors){
        if(a.second == ADeletedID){
            vtkSmartPointer<vtkActor> actor = a.first;
            m_vtk_renderer->RemoveActor(actor);
            a.first->Delete();
        }
    }
    m_surf_to_colors.erase(ADeletedID);
    m_vtk_window->Render();
    m_vtk_actors[0]->PickableOn();
}
/*-----*/
void GraphicView::showAxis(double ACoords[3][3]) {

    // Create the polydata where we will store all the geometric data
    vtkSmartPointer<vtkPolyData> linesPolyData =
        vtkSmartPointer<vtkPolyData>::New();

    double resize = (std::sqrt((m_vtk_window->GetSize()[0])*(m_vtk_window->GetSize()[0])+(m_vtk_window->GetSize()[1])*(m_vtk_windo
w->GetSize()[1])))*0.01;

    // Create a vtkPoints container and store the points in it
    vtkSmartPointer<vtkPoints> pts =
        vtkSmartPointer<vtkPoints>::New();
    pts->InsertNextPoint(ACoords[0]);

```

```

pts->InsertNextPoint (ACoords[1]);
pts->InsertNextPoint (ACoords[2]);

// Add the points to the polydata container
linesPolyData->SetPoints (pts);

// Create the first line (between Origin and P0)
vtkSmartPointer<vtkLine> line0 =
    vtkSmartPointer<vtkLine>::New();
line0->GetPointIds()->SetId(0, 0); // the second 0 is the index of the Origin in linesPolyData's points
line0->GetPointIds()->SetId(1, 1); // the second 1 is the index of P0 in linesPolyData's points

// Create the second line (between Origin and P1)
vtkSmartPointer<vtkLine> line1 =
    vtkSmartPointer<vtkLine>::New();
line1->GetPointIds()->SetId(0, 0); // the second 0 is the index of the Origin in linesPolyData's points
line1->GetPointIds()->SetId(1, 2); // 2 is the index of P1 in linesPolyData's points

// Create a vtkCellArray container and store the lines in it
vtkSmartPointer<vtkCellArray> lines =
    vtkSmartPointer<vtkCellArray>::New();
lines->InsertNextCell (line0);
lines->InsertNextCell (line1);

// Add the lines to the polydata container
linesPolyData->SetLines (lines);

// Create two colors - one for each line
unsigned char red[3] = {255, 0, 0};
unsigned char green[3] = {0, 255, 0};

// Create a vtkUnsignedCharArray container and store the colors in it
vtkSmartPointer<vtkUnsignedCharArray> colors =
    vtkSmartPointer<vtkUnsignedCharArray>::New();
colors->SetNumberOfComponents (3);
colors->InsertNextTypedTuple (red);
colors->InsertNextTypedTuple (green);

// Color the lines.
// SetScalars() automatically associates the values in the data array passed as parameter
// to the elements in the same indices of the cell data array on which it is called.
// This means the first component (red) of the colors array
// is matched with the first component of the cell array (line 0)
// and the second component (green) of the colors array
// is matched with the second component of the cell array (line 1)
linesPolyData->GetCellData()->SetScalars (colors);

// Setup the visualization pipeline
vtkSmartPointer<vtkPolyDataMapper> mapper =
    vtkSmartPointer<vtkPolyDataMapper>::New();
mapper->SetInputData (linesPolyData);

vtkSmartPointer<vtkActor> actor =
    vtkSmartPointer<vtkActor>::New();
actor->SetMapper (mapper);
actor->GetProperty()->SetLineWidth (4);

m_vtk_renderer->AddActor (actor);
m_vtk_window->Render();
}

/*-----*/
void GraphicView::removeAxis() {
    vtkSmartPointer<vtkActor> last_act = m_vtk_renderer->GetActors()->GetLastItem();
    m_vtk_renderer->RemoveActor (last_act);
    m_vtk_window->Render();
}

/*-----*/
void GraphicView::setModeDomain() {
    m_mode = 0;
    setDualVisibilityON();
    m_vtk_actors[0]->GetMapper()->ScalarVisibilityOff();
    m_vtk_actors[0]->GetProperty()->SetDiffuseColor (0.5,0.5,0.5);
    for (auto a : m_vtk_tets_actors) {
        std::vector<double> colors;
        colors.resize(3);
        colors = m_surf_to_colors[a.second];
        a.first->GetProperty()->SetDiffuseColor (colors[0],colors[1],colors[2]);
        a.first->VisibilityOn();
    }
    for (auto a : m_vtk_surface_actors) {
        std::vector<double> colors;
        colors.resize(3);
        colors = m_surf_to_colors[a.second];
        a.first->GetProperty()->SetDiffuseColor (colors[0],colors[1],colors[2]);
        a.first->VisibilityOn();
    }
    refresh();
}

```



```

/*-----*/
void GraphicView::setModeDual() {
    m_mode = 1;
}
/*-----*/
void GraphicView::viewBlocks() {
    //update();
    m_vtk_actors[0]->VisibilityOff();
    m_vtk_actors[0]->GetMapper()->SetScalarModeToUseCellData();
    m_vtk_actors[0]->GetMapper()->SetColorModeToMapScalars();
    m_vtk_actors[0]->GetMapper()->ScalarVisibilityOn();

    for(auto a : m_vtk_actors){
        a->VisibilityOff();
    }

    m_vtk_actors[1]->VisibilityOn();
    m_vtk_actors[1]->GetProperty()->SetRepresentationToSurface();
    m_vtk_actors[1]->GetProperty()->EdgeVisibilityOn();

    m_block_mode = true;
}
/*-----*/
void GraphicView::viewSurface() {
    if(surface_mode == 0) {
        for(auto a : m_vtk_tets_actors){
            a.first->VisibilityOff();
        }
        for(auto a : m_vtk_surface_actors){
            a.first->VisibilityOn();
        }
        surface_mode = 1;
    }else{
        for(auto a : m_vtk_tets_actors){
            a.first->VisibilityOn();
        }
        for(auto a : m_vtk_surface_actors){
            a.first->VisibilityOff();
        }
        surface_mode = 0;
    }

    m_vtk_window->Render();
}
/*-----*/
void GraphicView::createPlanSurface(int ASheetID, MedusaGrid* ASurf) {

    double colors[3];

    for(auto c : m_surf_to_colors){
        if(c.first == ASheetID){
            colors[0] = c.second[0];
            colors[1] = c.second[1];
            colors[2] = c.second[2];
        }
    }

    vtkDataSetMapper *mapper = vtkDataSetMapper::New();
    mapper->SetInputData(ASurf->grid());
    vtkSmartPointer<vtkActor> actor = vtkActor::New();
    actor->SetMapper(mapper);
    actor->GetProperty()->SetDiffuseColor(colors);
    m_vtk_surface_actors.insert(std::pair<vtkSmartPointer<vtkActor>,int>(actor,ASheetID));
    if(surface_mode == 0){
        actor->VisibilityOff();
    }

    m_vtk_renderer->AddActor(actor);

    m_vtk_window->Render();
}
/*-----*/
void GraphicView::toggleBlocks() {
    if(!m_block_mode){
        m_vtk_actors[0]->VisibilityOff();
        m_vtk_actors[1]->VisibilityOn();
        m_block_mode = true;
    } else{
        m_vtk_actors[0]->VisibilityOn();
        m_vtk_actors[1]->VisibilityOff();
        m_block_mode = false;
    }
}
/*-----*/
void GraphicView::buildSingGraph(std::vector<vtkIdType> AIDs) {

    if(AIDs.empty()) {
        return;
    }

    m_sinGraph_visibility = 1;

    vtkSmartPointer<vtkNamedColors> colors =

```

```

    vtkSmartPointer<vtkNamedColors>::New();

    vtkSmartPointer<vtkIdTypeArray> ids =
        vtkSmartPointer<vtkIdTypeArray>::New();
    ids->SetNumberOfComponents(1);
    for(auto id:AIDs)
        ids->InsertNextValue(id);

    vtkSmartPointer<vtkSelectionNode> selectionNode =
        vtkSmartPointer<vtkSelectionNode>::New();
    selectionNode->SetFieldType(vtkSelectionNode::CELL);
    selectionNode->SetContentType(vtkSelectionNode::INDICES);
    selectionNode->SetSelectionList(ids);

    vtkSmartPointer<vtkSelection> selection =
        vtkSmartPointer<vtkSelection>::New();
    selection->AddNode(selectionNode);

    vtkSmartPointer<vtkExtractSelection> extractSelection =
        vtkSmartPointer<vtkExtractSelection>::New();
    extractSelection->SetInputData(0, MedusaBackend::getInstance()->grids()[0]->grid());
    extractSelection->SetInputData(1, selection);
    extractSelection->Update();

    // In selection
    vtkSmartPointer<vtkUnstructuredGrid> selected =
        vtkSmartPointer<vtkUnstructuredGrid>::New();
    selected->ShallowCopy(extractSelection->GetOutput());

    vtkDataSetMapper *mapper_selection = vtkDataSetMapper::New();
    mapper_selection->SetInputData(selected);
    vtkSmartPointer<vtkActor> actor_selection = vtkActor::New();
    actor_selection->SetMapper(mapper_selection);
    actor_selection->GetProperty()->EdgeVisibilityOff();
    actor_selection->GetProperty()->SetColor(colors->GetColor3d("Red").GetData());

    actor_selection->GetProperty()->SetLineWidth(5);
    actor_selection->GetProperty()->SetPointSize(5);
    actor_selection->PickableOff();

    m_singGraph = actor_selection;

    m_vtk_actors.push_back(actor_selection);
    m_vtk_renderer->AddActor(actor_selection);

    m_vtk_window->Render();
}
/*-----*/
void GraphicView::resetSingLine() {
    for(auto line : m_vtk_sing_actors) {
        m_vtk_renderer->RemoveActor(line.first);
        line.first->Delete();
    }
    m_vtk_sing_actors.clear();
}
/*-----*/
void GraphicView::buildSingLine(int AID, std::vector<vtkIdType> AIDs) {

    vtkSmartPointer<vtkNamedColors> colors =
        vtkSmartPointer<vtkNamedColors>::New();

    vtkSmartPointer<vtkIdTypeArray> ids =
        vtkSmartPointer<vtkIdTypeArray>::New();
    ids->SetNumberOfComponents(1);
    for(auto id:AIDs)
        ids->InsertNextValue(id);

    vtkSmartPointer<vtkSelectionNode> selectionNode =
        vtkSmartPointer<vtkSelectionNode>::New();
    selectionNode->SetFieldType(vtkSelectionNode::CELL);
    selectionNode->SetContentType(vtkSelectionNode::INDICES);
    selectionNode->SetSelectionList(ids);

    vtkSmartPointer<vtkSelection> selection =
        vtkSmartPointer<vtkSelection>::New();
    selection->AddNode(selectionNode);

    vtkSmartPointer<vtkExtractSelection> extractSelection =
        vtkSmartPointer<vtkExtractSelection>::New();
    extractSelection->SetInputData(0, MedusaBackend::getInstance()->grids()[0]->grid());
    extractSelection->SetInputData(1, selection);
    extractSelection->Update();

    // In selection
    vtkSmartPointer<vtkUnstructuredGrid> selected =

```

```

        vtkSmartPointer<vtkUnstructuredGrid>::New();
selected->ShallowCopy(extractSelection->GetOutput());

vtkDataSetMapper *mapper_selection = vtkDataSetMapper::New();
mapper_selection->SetInputData(selected);
vtkSmartPointer<vtkActor> actor_selection = vtkActor::New();
actor_selection->SetMapper(mapper_selection);
actor_selection->GetProperty()->EdgeVisibilityOff();
actor_selection->GetProperty()->SetColor(colors->GetColor3d("Red").GetData());

actor_selection->GetProperty()->SetLineWidth(5);
actor_selection->GetProperty()->SetPointSize(5);
actor_selection->PickableOff();

m_vtk_sing_actors.emplace(actor_selection, AID);
m_vtk_renderer->AddActor(actor_selection);

m_vtk_window->Render();
}
/*-----*/
void GraphicView::toggleSingGraph() {
    if(m_sinGraph_visibility){
        m_sinGraph->VisibilityOff();
        m_sinGraph_visibility = false;
    }else{
        m_sinGraph->VisibilityOn();
        m_sinGraph_visibility = true;
    }
}
/*-----*/
void GraphicView::surfacePicked(std::vector<vtkIdType> AIDs) {
    vtkSmartPointer<vtkNamedColors> colors =
        vtkSmartPointer<vtkNamedColors>::New();

    vtkSmartPointer<vtkIdTypeArray> ids =
        vtkSmartPointer<vtkIdTypeArray>::New();
    ids->SetNumberOfComponents(1);
    for(auto id:AIDs)
        ids->InsertNextValue(id);

    vtkSmartPointer<vtkSelectionNode> selectionNode =
        vtkSmartPointer<vtkSelectionNode>::New();
    selectionNode->SetFieldType(vtkSelectionNode::CELL);
    selectionNode->SetContentType(vtkSelectionNode::INDICES);
    selectionNode->SetSelectionList(ids);

    vtkSmartPointer<vtkSelection> selection =
        vtkSmartPointer<vtkSelection>::New();
    selection->AddNode(selectionNode);

    vtkSmartPointer<vtkExtractSelection> extractSelection =
        vtkSmartPointer<vtkExtractSelection>::New();
    extractSelection->SetInputData(0, MedusaBackend::getInstance()->grids()[0]->grid());
    extractSelection->SetInputData(1, selection);
    extractSelection->Update();

    // In selection
    vtkSmartPointer<vtkUnstructuredGrid> selected =
        vtkSmartPointer<vtkUnstructuredGrid>::New();
    selected->ShallowCopy(extractSelection->GetOutput());

    vtkDataSetMapper *mapper_selection = vtkDataSetMapper::New();
    mapper_selection->SetInputData(selected);
    vtkSmartPointer<vtkActor> actor_selection = vtkActor::New();
    actor_selection->SetMapper(mapper_selection);
    actor_selection->GetProperty()->EdgeVisibilityOff();
    actor_selection->GetProperty()->SetColor(colors->GetColor3d("Red").GetData());

    actor_selection->GetProperty()->SetLineWidth(5);
    actor_selection->GetProperty()->SetPointSize(5);
    actor_selection->PickableOff();

    m_vtk_selection_actors.push_back(actor_selection);

    m_vtk_renderer->AddActor(actor_selection);

    m_vtk_window->Render();
}
/*-----*/
void GraphicView::toggleBRep() {
    //m_vtk_actors[0]->VisibilityOff();

```

```

m_vtk_actors[1]->VisibilityOn();
m_vtk_actors[1]->PickableOff();
//m_vtk_actors[0]->PickableOn();

MedusaGrid* g = MedusaBackend::getInstance()->grids()[1];

vtkDataArray *array = g->grid()->GetCellData()->GetArray("BND_Color");

MedusaBackend::getInstance()->grids()[1]->grid()->GetCellData()->SetScalars(array);

int nbSurfaces = MedusaBackend::getInstance()->grids()[1]->getNbGeomSurfs();

vtkDataSetMapper *mapper = vtkDataSetMapper::New();
mapper->SetInputData(g->grid());
mapper->SetScalarModeToUseCellData();
mapper->SetColorModeToMapScalars();
mapper->ScalarVisibilityOn();
mapper->SetScalarRange(0,nbSurfaces);

mapper->Update();

m_vtk_actors[1]->SetMapper(mapper);
}
/*-----*/
void GraphicView::textMode(int AMode) {
    // Setup the text and add it to the renderer

    switch (AMode) {
        case 0:
            m_textActor->SetInput ( "Input mode" );
            break;
        case 1:
            m_textActor->SetInput ( "Frame mode" );
            break;
        case 2:
            m_textActor->SetInput ( "Geom mode" );
            break;
        case 3:
            m_textActor->SetInput ( "Dublo mode" );
            break;
    }
    m_vtk_window->Render();
}
/*-----*/
void GraphicView::hideSetOfTets() {
    m_vtk_tets_set->VisibilityOff();
}
/*-----*/
void GraphicView::showPoints(std::vector<std::vector<double>> coords) {
    vtkSmartPointer<vtkNamedColors> colors =
        vtkSmartPointer<vtkNamedColors>::New();

    for(auto coord : coords){
        vtkSmartPointer<vtkSphereSource> sphereSource =
            vtkSmartPointer<vtkSphereSource>::New();
        sphereSource->SetCenter(coord[0],coord[1],coord[2]);
        sphereSource->SetRadius(0.25);

        vtkSmartPointer<vtkPolyDataMapper> mapper =
            vtkSmartPointer<vtkPolyDataMapper>::New();

        mapper->SetInputConnection(sphereSource->GetOutputPort());
        vtkSmartPointer<vtkActor> actor =
            vtkSmartPointer<vtkActor>::New();
        actor->SetMapper(mapper);
        actor->GetProperty()->SetColor(colors->GetColor3d("Red").GetData());

        m_vtk_renderer->AddActor(actor);
        m_vtk_actors.push_back(actor);
    }

    m_vtk_window->Render();
}
/*-----*/
void GraphicView::resetDual() {
    for(auto a : m_vtk_tets_actors){
        m_vtk_renderer->RemoveActor(a.first);
        a.first->Delete();
    }

    m_vtk_tets_actors.clear();

    for(auto a : m_vtk_surface_actors){
        m_vtk_renderer->RemoveActor(a.first);
        a.first->Delete();
    }
}

```

```
    m_vtk_surface_actors.clear();  
  
}
```

```
/*-----*/
#ifndef GMDS_GRAPHICVIEW_H
#define GMDS_GRAPHICVIEW_H
/*-----*/
#include <medusa/view/View.h>
/*-----*/
#include <vtkSmartPointer.h>
/*-----*/
#include <vector>
#include <string>
#include <vtkDataSetMapper.h>
#include <map>
#include <medusa/model/MedusaGrid.h>
#include <vtkTextActor.h>
#include <vtkTextProperty.h>
#include <vtkSelection.h>

/*-----*/
class vtkRenderer;
class vtkRenderWindow;
class vtkActor;
class vtkRenderWindowInteractor;
/*-----*/
namespace medusa{
    class PickingListener;
    class MouseInteractorStyle;
    class GraphicView: public View{
    public:
        enum ViewType{
            View3D, View2DX, View2DY, View2DZ
        };

        /**
         * @brief constructor. When a Graphic view is created, the picking object
         * is created too.
         */
        GraphicView(const std::string& AName, GraphicView::ViewType AType);
        virtual ~GraphicView();
        void selectVTKCell(int ADIm, vtkIdType ACellID);
        void refresh();
        void update();
        void setVisibleON();
        void setVisibleOFF();
        void start();

        void textMode(int AMode);

        void showSetOfTets(std::vector<vtkIdType> AIDs);

        //Visualization operations
        void wireframe();
        void solid();

        void opacity();
        void setOpacityON();
        void setOpacityOFF();

        //FF operations
        void toggleSingGraph();
        void buildSingGraph(std::vector<vtkIdType> AIDs);
        void buildSingLine(int AID, std::vector<vtkIdType> AIDs);
        void resetSingLine();

        void unlockLines();
        int selectLine(const vtkSmartPointer<vtkActor> ADeletedActor);

        //Dual operations
        void createSurface(std::vector<vtkIdType> AIDs, int ASheetID);
        void createPlanSurface(int ASheetID, MedusaGrid* ASurf);

        void viewSurface();

        void updateDual(int ANbZones);
        void setDualVisibilityON();
        void setDualVisibilityOFF();

        void unlockSheet();

        int removeActor(vtkSmartPointer<vtkActor> ADeletedActor);
        void removeActor(int ADeletedID);

        void showAxis(double ACoords[3][3]);
        void removeAxis();

        void setModeDomain();
        void setModeDual();

        void updateMesh();

        void viewBlocks();
        void toggleBlocks();
    };
}
```

```

//Geom operation
void surfacePicked(std::vector<vtkIdType> AIDs);
void removeLastSelection();
void removeSelection();

void updateCut();
void toggleCutOpacity();

void toggleBRep();

void hideSetOfTets();
void addTetSetActor();

void showPoints(std::vector<std::vector<double>> coords);

void resetDual();

private:
std::string m_name;
bool m_visible;
ViewType m_type;
vtkSmartPointer<vtkRenderer> m_vtk_renderer;
vtkSmartPointer<vtkRenderWindow> m_vtk_window;
vtkSmartPointer<vtkRenderWindowInteractor> m_vtk_interactor;
vtkSmartPointer<MouseInteractorStyle> m_mouse;
std::vector<vtkSmartPointer<vtkActor> > m_vtk_actors;
std::map<vtkSmartPointer<vtkActor>,int> m_vtk_surface_actors;
std::map<vtkSmartPointer<vtkActor>,int> m_vtk_tets_actors;
std::map<vtkSmartPointer<vtkActor>,int> m_vtk_sing_actors;

vtkSmartPointer<vtkActor> m_vtk_tets_set;
std::vector<vtkSmartPointer<vtkActor> > m_vtk_sets_actors;

vtkSmartPointer<vtkActor> m_vtk_cut_actor;
std::vector<vtkSmartPointer<vtkActor>> m_vtk_selection_actors;

vtkSmartPointer<vtkDataSetMapper> selectedMapper;
vtkSmartPointer<vtkActor> selectedActor;
vtkDataSetMapper *m_mapper_dual;
vtkSmartPointer<vtkActor> m_singGraph;

vtkSmartPointer<vtkTextActor> m_textActor;

std::map<int, std::vector<double>> m_surf_to_colors;

void wireframeAll();

bool m_opacity = true;

int m_op_cut = 1;

int m_nb_zones = 0;

int m_nb_surfaces = 0;

//The mode of the view:
/* 0 = domain editing : show the domain tet mesh without data colormap + the dual surfaces with colors
 * 1 = dual structure editing : show the tet mesh with dual data color, dual zones + dual surfaces in red
 */
int m_mode = 0;

int surface_mode = 0;

bool m_block_mode = false;

/*
 * -1 not initialized, 0 hidden, 1 visible
 */
int m_singGraph_visibility = -1;

int test = 0;

};
}
/*-----*/
#endif //GMD5_GRAPHICVIEW_H
/*-----*/

```

```
/*-----*/
#include <medusa/Medusa.h>
/*-----*/
#include <gmds/Utils/Exception.h>
/*-----*/
#include <iostream>
/*-----*/
using namespace medusa;
/*-----*/
int main(int argc, char* argv[])
{
    std::cout<<"Medusa (Mesh Blocking)"<<std::endl;

    //=====
    std::string fIn;

    if (argc < 2)
        //throw gmds::GMDSException("No file loaded");
        std::cout<<"No file loaded "<<std::endl;

    if (argc >= 2) {
        fIn = std::string(argv[1]);
        std::cout << "INPUT FILE: " << fIn << std::endl;
    }

    Medusa m(Medusa::DUAL_BLOCKING);
    m.newGraphicView("Medusa 1", GraphicView::View3D);
    m.newTextView();
    if(fIn.length() == 0){
        std::cout<<"INPUT FILE : "<<std::endl;
        std::cin >> fIn;
    }
    m.setMode(Medusa::GEOMETRY);
    m.load(fIn);
    m.launch();

    return 0;
}
/*-----*/
```



```
/*-----*/
#include <gtest/gtest.h>
/*-----*/
// Files containing the different test suites to launch
#include "MedusaTestSuite.h"
/*-----*/
int main(int argc, char ** argv) {
    ::testing::InitGoogleTest(&argc, argv);
    return RUN_ALL_TESTS();
}
/*-----*/
```

```

/*-----*/
#include <medusa/control/MediatorControl.h>

#include <medusa/model/MedusaBackEnd.h>
#include <medusa/control/MediatorExplorerPolicy.h>
/*-----*/
#include <iostream>
/*-----*/
#include <gmds/utils/Exception.h>
#include <medusa/control/MedusaCommandGenerate.h>
#include <medusa/control/MediatorDuBloPolicy.h>
/*-----*/
using namespace medusa;
using namespace gmds;
/*-----*/
MediatorControl* MediatorControl::m_instance=NULL;
/*-----*/
MediatorControl* MediatorControl::getInstance() {return m_instance;}
/*-----*/
void MediatorControl::initialize() {
    if(m_instance==NULL)
        m_instance=new MediatorControl();
}/*-----*/
MediatorControl::MediatorControl():m_policy(new MediatorExplorerPolicy()) {}
/*-----*/
MediatorControl::~MediatorControl() {}
/*-----*/
void MediatorControl::setPolicy(medusa::MediatorPolicyItf *APolicy) {
    if(APolicy==NULL)
        throw GMDSException("Error, null policy!!!");
    m_policy=APolicy;
}
/*-----*/
void MediatorControl::update() {
    std::cout<<"GlobalEventListener: receive a notification from the back end"<<std::endl;
}
/*-----*/
void MediatorControl::addCollaborator(GraphicView *AView) {
    m_graphic_collaborator.push_back(AView);
}
/*-----*/
void MediatorControl::addCollaborator(TextView *AView) {
    m_text_collaborator.push_back(AView);
}
/*-----*/
void MediatorControl::select(int ADIm, vtkIdType ACellID) {

    //if(MedusaBackend::getInstance()->grids().size()!=1)
    //throw GMDSException("Warning, picking is only available for a single mesh");

    MedusaCommandPick* c = new MedusaCommandPick(m_policy, m_graphic_collaborator[0],m_text_collaborator[0],ADIm, ACellID);
    MedusaCommand::State sc = c->execute();
    if(sc == MedusaCommand::SUCCESE) {
        m_commands.push_back(c);
    }else{
        std::cout << "FAILED command : [pick]" << std::endl;
    }
}
/*-----*/
void MediatorControl::help() {
    for(auto v:m_text_collaborator){
        m_policy->help(v);
    }
}
/*-----*/
void MediatorControl::generate() {
    MedusaCommandGenerate *c = new MedusaCommandGenerate(m_policy, m_graphic_collaborator[0], m_text_collaborator[0]);
    MedusaCommand::State sc = c->execute();
    if (sc == MedusaCommand::SUCCESE){
        m_commands.push_back(c);
    }else{
        std::cout << "FAILED command : [generate]" << std::endl;
    }
}
/*-----*/
void MediatorControl::toggleDual() {
    if(m_dual_state == -1){
        m_text_collaborator[0]->dualError();
    }
    else if(m_dual_state == 0) {
        m_graphic_collaborator[0]->setDualVisibilityON();
        m_dual_state = 1;
    }else{
        m_graphic_collaborator[0]->setDualVisibilityOFF();
        m_dual_state = 0;
    }
}
/*-----*/
void MediatorControl::wireframe(){
    for(auto v:m_graphic_collaborator){
        v->wireframe();
    }
}

```

```
    }
}
/*-----*/
void MediatorControl::opacity() {
    m_policy->opacity(m_graphic_collaborator[0]);
}
/*-----*/
void MediatorControl::remove() {
    m_policy->remove(m_graphic_collaborator[0]);
}
/*-----*/
int MediatorControl::removeActor(vtkSmartPointer<vtkActor> ADeletedActor) {

    //=====
    //Ici il faut descendre l'appel aux policy
    //=====

    return m_graphic_collaborator[0]->selectLine(ADeletedActor);
    return m_graphic_collaborator[0]->removeActor(ADeletedActor);
}
/*-----*/
void MediatorControl::removeItem(int AItemID) {
    m_text_collaborator[0]->remove(AItemID);
    m_policy->removeItem(AItemID, m_graphic_collaborator[0]);
}
/*-----*/
void MediatorControl::surfaceMode() {
    m_policy->surfaceMode(m_graphic_collaborator[0]);
}
/*-----*/
void MediatorControl::undo() {
    if(!m_commands.empty()) {
        MedusaCommand *c = m_commands.back();
        c->undo();
        m_commands.pop_back();
    }
}
/*-----*/
void MediatorControl::singularityGraphToggle() {
    m_policy->singularityGraphToggle(m_graphic_collaborator[0]);
}
/*-----*/
void MediatorControl::color() {
    m_policy->color(m_graphic_collaborator[0]);
}
/*-----*/
void MediatorControl::changePolicy() {

    m_policy = new medusa::MediatorDubloPolicy;
    MedusaBackend::getInstance()->modeBlocks();
    m_graphic_collaborator[0]->textMode(3);
    //m_graphic_collaborator[0]->updateCut();
}
/*-----*/
```

```

/*-----*/
#ifndef GMDS_MEDIATORCONTROL_H
#define GMDS_MEDIATORCONTROL_H
/*-----*/
#include <medusa/view/MedusaObserver.h>
#include <medusa/view/GraphicView.h>
#include <medusa/view/TextView.h>
#include <medusa/control/MediatorPolicyItf.h>
/*-----*/
#include <vtkType.h>
#include "MedusaCommand.h"
#include "MedusaCommandPick.h"
/*-----*/
namespace medusa{
    class MediatorControl:public MedusaObserver{
    public:

        static MediatorControl* getInstance();
        static void initialize();
        void update();

        void addCollaborator(GraphicView* AView);
        void addCollaborator(TextView* AView);

        void setPolicy(MediatorPolicyItf* APolicy);

        void select(int ADIm, vtkIdType ACellID);
        void help();

        void generate();
        void toggleDual();

        void wireframe();

        void opacity();

        void remove();

        int removeActor(vtkSmartPointer<vtkActor> ADeletedActor);

        void removeItem(int AItemID);

        void surfaceMode();
        void finalMesh();
        void undo();

        void color();

        void singularityGraphToggle();

        void changePolicy();

    protected:

        MediatorControl();
        virtual ~MediatorControl();

    private:
        static MediatorControl* m_instance;

        MediatorPolicyItf* m_policy;
        std::vector<GraphicView*> m_graphic_collaborator;
        std::vector<TextView*> m_text_collaborator;
        std::vector<MedusaCommand*> m_commands;

        //flag for the visualisation of the dual, -1 the dual isnt generated yet, 0 the dual is hidden and 1 the dual is visible
        int m_dual_state = -1;

    };
}
/*-----*/
#endif //GMDS_MEDIATORCONTROL_H
/*-----*/

```

```

/*-----*/
#include <medusa/control/MediatorDubloPolicy.h>
/*-----*/
#include <medusa/view/GraphicView.h>
#include <medusa/view/TextView.h>
#include <gmds/utils/Exception.h>
#include <medusa/model/MedusaBackend.h>
/*-----*/
using namespace medusa;
/*-----*/
MediatorDubloPolicy::MediatorDubloPolicy() {}
/*-----*/
MediatorDubloPolicy::~MediatorDubloPolicy() {}
/*-----*/
int MediatorDubloPolicy::
selectVTKCell(TextView *AView, const int ADim, const vtkIdType AID)
{
    AView->selectVTKCell(ADim,AID);
    return AView->getInputAxis();
}
/*-----*/
int MediatorDubloPolicy::
selectVTKCell(GraphicView *AView, const int ADim, const vtkIdType AID, int AAxis)
{
    double coords[3][3];
    int sheet_ID = 0;
    if(MedusaBackend::getInstance()->getFrameAxis(coords,AID)) {

        AView->showAxis(coords);
        char axis = '0';
        int axis_index = 0;
        bool wrong_answer = true;
        while (wrong_answer) {
            std::cout
                << "Select dual surface normal direction : '1' for red, '2' for green or '3' to create a boundary surface"
                << std::endl;
            cin >> axis;
            switch (axis) {
                case '1':
                    wrong_answer = false;
                    axis_index = 1;
                    break;
                case '2':
                    wrong_answer = false;
                    axis_index = 2;
                    break;
                case '3':
                    wrong_answer = false;
                    axis_index = 3;
                    break;
                case 'q':
                    wrong_answer = false;
                    axis_index = 0;
                    break;
                default:
                    std::cout << "Wrong input, please retry." << std::endl;
                    cin.clear();
            }
        }
        std::cout << "Test" << std::endl;
        AView->removeAxis();
        std::vector<vtkIdType> surf;

        if (axis_index == 3) {
            surf = MedusaBackend::getInstance()->createBoundary(ADim, AID, sheet_ID);
            AView->updateMesh();
        } else {
            surf = MedusaBackend::getInstance()->createSurface(ADim, AID, sheet_ID, axis_index);
        }
        if (sheet_ID != -1) {
            AView->createSurface(surf, sheet_ID);
            AView->createPlanSurface(sheet_ID, MedusaBackend::getInstance()->grids().back());
        }
    }
    else{
        sheet_ID = -1;
    }

    return sheet_ID;
}
/*-----*/
void MediatorDubloPolicy::
help(TextView *AView)
{AView->help();}
/*-----*/
void MediatorDubloPolicy::
generate(GraphicView *AView)
{
    if(m_mode == 0) {
        AView->updateDual(MedusaBackend::getInstance()->generateDual());
        m_mode = 1;
    } else if(m_mode == 1){
        MedusaBackend::getInstance()->generateBlocks();
        AView->viewBlocks();
        m_mode = 2;
    }
}

```

```

    }
}
/*-----*/
void MediatorDubloPolicy::
generate(TextView *AView)
{
    if(m_mode == 0) {
        AView->generateDual();
        //m_mode = 1;
    } else if(m_mode == 1){
        //m_mode = 2;
    } else if(m_mode == 2){
        std::cout<<"Blocks already generated"<<std::endl;
    }
}
/*-----*/
void MediatorDubloPolicy::remove(GraphicView *AView) {
    AView->unlockSheet();
}
/*-----*/
void MediatorDubloPolicy::
removeItem(int AID, GraphicView *AView){
    MedusaBackend::getInstance()->removeSheet(AID);
    AView->removeActor(AID);
}
/*-----*/
void MediatorDubloPolicy::
removeItem(int AID, TextView *AView)
{MedusaBackend::getInstance()->removeSheet(AID);}
/*-----*/
void MediatorDubloPolicy::
showAxis(GraphicView *AView, vtkIdType ACellID)
{
    //create a table to store the coordinates of the frame vectors
    //coords[0] is the center of the cell and coords[1,2,3] the three directions of the frame chart in the mesh cell
    double coords[3][3];
    MedusaBackend::getInstance()->getFrameAxis(coords, ACellID);
    AView->showAxis(coords);
}
/*-----*/
void MediatorDubloPolicy::surfaceMode(GraphicView *AView) {
    AView->viewSurface();
}
/*-----*/
void MediatorDubloPolicy::undoGenerate(GraphicView *AView) {
    if(m_mode == 1) {
        MedusaBackend::getInstance()->resetDual();
        AView->resetDual();
        AView->setModeDomain();
        m_mode = 0;
    } else if(m_mode == 2){
        MedusaBackend::getInstance()->resetBlock();
        AView->toggleBlocks();
        m_mode = 1;
    }
}
/*-----*/
void MediatorDubloPolicy::singularityGraphToggle(GraphicView *AView) {
    AView->toggleSingGraph();
}
/*-----*/
void MediatorDubloPolicy::opacity(GraphicView *AView) {
    AView->opacity();
}
/*-----*/
void MediatorDubloPolicy::color(GraphicView *AView) {
    if(m_mode == 1) {
        int nbZones;
        std::cout<<"Press \"e\" to erase wrong dual regions or \"c\" to continue with this dual structure"<<std::endl;
        char input;
        cin>>input;
        if(input == 'e'){
            nbZones = MedusaBackend::getInstance()->correctDualRegions();
            AView->updateDual(nbZones);
        } else if (input == 'c') {
            nbZones = MedusaBackend::getInstance()->reGenerateDual();
        } else{
            throw std::string("Error! wrong input");
        }

        AView->updateDual(nbZones);
        cin.clear();
    }
}

```

```
/*-----*/
#ifndef GMDS_MEDIATORDUBLOPOLICY_H
#define GMDS_MEDIATORDUBLOPOLICY_H
/*-----*/
#include <medusa/control/MediatorPolicyItf.h>
/*-----*/

namespace medusa {
    class MediatorDubloPolicy: public MediatorPolicyItf {

    public:
        MediatorDubloPolicy();
        virtual ~MediatorDubloPolicy();

        /*
         * Pick a cell to start the dual sheet creation
         */
        int selectVTKCell(GraphicView* AView,
                        const int ADim, const vtkIdType AID, int AAxis);
        int selectVTKCell(TextView* AView,
                        const int ADim, const vtkIdType AID);
        void help(TextView* AView);

        void remove(GraphicView *AView);

        /*
         * First, generate the dual structure, if the dual structure is already generated, generate the block structure instead
         */
        void generate(GraphicView *AView);
        void generate(TextView *AView);

        /*
         * Remove a dual sheet of ID ASheetID
         */
        void removeItem(int ASheetID, GraphicView *AView);
        void removeItem(int ASheetID, TextView *AView);

        /*
         * Show the 3 frame field direction at the cell of ID ACellID in the graphic view AView
         */
        void showAxis(GraphicView *AView, vtkIdType ACellID);

        /*
         * Toggle the surface mode for the dual sheet, instead of viewing tetras, only the plane used to create the sheet is shown
         */
        void surfaceMode(GraphicView *AView);

        /*
         * Undo the last generate action done
         */
        void undoGenerate(GraphicView *AView);

        void singularityGraphToggle(GraphicView *AView);

        void opacity(GraphicView *AView);

        void color(GraphicView *AView);

    protected:
        int m_mode = 0;
    };
}
/*-----*/
#endif //GMDS_MEDIATORDUBLOPOLICY_H
/*-----*/
```

```
/*-----*/
#include <medusa/control/MediatorExplorerPolicy.h>
/*-----*/
#include <medusa/view/GraphicView.h>
#include <medusa/view/TextView.h>
/*-----*/
using namespace medusa;
/*-----*/
MediatorExplorerPolicy::MediatorExplorerPolicy() {}
/*-----*/
MediatorExplorerPolicy::~MediatorExplorerPolicy() {}
/*-----*/
int MediatorExplorerPolicy::
selectVTKCell(TextView *AView, const int ADim, const vtkIdType AID)
{
    AView->selectVTKCell(ADim,AID); }
/*-----*/
int MediatorExplorerPolicy::
selectVTKCell(GraphicView *AView, const int ADim, const vtkIdType AID, int AAxis)
{AView->selectVTKCell(ADim,AID);
    return 0;}
/*-----*/
void MediatorExplorerPolicy::
help(TextView *AView)
{AView->help();}
/*-----*/
void MediatorExplorerPolicy::remove(GraphicView *AView) {
    std::cout<<"Not implemented yet"<<std::endl;
}
/*-----*/
void MediatorExplorerPolicy::removeItem(int AID, GraphicView *AView) {
    std::cout<<"Not implemented yet"<<std::endl;
}
/*-----*/
void MediatorExplorerPolicy::removeItem(int AID, TextView *AView) {
    std::cout<<"Not implemented yet"<<std::endl;
}
/*-----*/
void MediatorExplorerPolicy::generate(GraphicView *AView) {
    std::cout<<"Not implemented yet"<<std::endl;
}
/*-----*/
void MediatorExplorerPolicy::generate(TextView *AView) {
    std::cout<<"Not implemented yet"<<std::endl;
}
/*-----*/
void MediatorExplorerPolicy::undoGenerate(GraphicView *AView) {
    std::cout<<"Not implemented yet"<<std::endl;
}
/*-----*/
void MediatorExplorerPolicy::showAxis(GraphicView *AView, vtkIdType ACellID) {
    std::cout<<"Not implemented yet"<<std::endl;
}
/*-----*/
void MediatorExplorerPolicy::surfaceMode(medusa::GraphicView *AView) {
    std::cout<<"Not implemented yet"<<std::endl;
}
/*-----*/
void MediatorExplorerPolicy::singularityGraphToggle(GraphicView *AView) {
    std::cout<<"Not implemented yet"<<std::endl;
}
/*-----*/
void MediatorExplorerPolicy::opacity(GraphicView *AView) {
    std::cout<<"Not implemented yet"<<std::endl;
}
/*-----*/
void MediatorExplorerPolicy::color(GraphicView *AView) {
    std::cout<<"Not implemented yet"<<std::endl;
}
}
```



```
/*-----*/
#ifndef GMDS_MEDIATOREXPLORERPOLICY_H
#define GMDS_MEDIATOREXPLORERPOLICY_H
/*-----*/
#include <medusa/control/MediatorPolicyItf.h>
/*-----*/

namespace medusa {
    class MediatorExplorerPolicy : public MediatorPolicyItf {

    public:
        MediatorExplorerPolicy();
        virtual ~MediatorExplorerPolicy();
        int selectVTKCell(GridView* AView,
                        const int ADim, const vtkIdType AID, int AAxis);
        int selectVTKCell(Textview* AView,
                        const int ADim, const vtkIdType AID);
        void help(Textview* AView);

        void remove(GridView *AView);

        void removeItem(int ASheetID, GridView *AView);
        void removeItem(int ASheetID, Textview *AView);

        void generate(GridView *AView);
        void generate(Textview *AView);

        void showAxis(GridView *AView, vtkIdType ACellID);

        void surfaceMode(GridView *AView);

        void undoGenerate(GridView *AView);

        void singularityGraphToggle(GridView *AView);

        void opacity(GridView *AView);

        void color(GridView *AView);
    };
}
/*-----*/
#endif //GMDS_MEDIATOREXPLORERPOLICY_H
/*-----*/
```

```

/*-----*/
#include "medusa/control/MediatorGeomPolicy.h"
/*-----*/
#include <medusa/view/GraphicView.h>
#include <medusa/view/TextView.h>
#include <medusa/model/MedusaBackend.h>
/*-----*/
using namespace medusa;
/*-----*/
MediatorGeomPolicy::MediatorGeomPolicy() {}
/*-----*/
MediatorGeomPolicy::~MediatorGeomPolicy() {}
/*-----*/
int MediatorGeomPolicy::
selectVTKCell(Textview *AView, const int ADim, const vtkIdType AID)
{AView->selectVTKCell(ADim,AID);}
/*-----*/
int MediatorGeomPolicy::
selectVTKCell(GraphicView *AView, const int ADim, const vtkIdType AID, int AAxis)
{
    std::vector<vtkIdType> ids;
    int surf_id = MedusaBackend::getInstance()->pickSurface(AID, ids);
    if(surf_id == -1)
        return -1;
    if(m_mode == 1){
        MedusaBackend::getInstance()->removeSurfFromSelection();
        MedusaBackend::getInstance()->unconstrainFrameTets(MedusaBackend::getInstance()->vtkToGMDSIDs(ids));
    }
    AView->surfacePicked(ids);

    return surf_id;
}
/*-----*/
void MediatorGeomPolicy::
help(Textview *AView)
{AView->help();}
/*-----*/
void MediatorGeomPolicy::remove(GraphicView *AView) {
    AView->unlockLines();
}
/*-----*/
void MediatorGeomPolicy::removeItem(int AID, GraphicView *AView) {
    if(m_mode == 1){

        std::vector<gmds::TCellID> offset =
            MedusaBackend::getInstance()->getOffset(1, MedusaBackend::getInstance()->pickSingLine(AID));

        AView->showSetOfTets(MedusaBackend::getInstance()->gmdsToVtkIDs(offset));

        char more = '0';
        bool again = true;
        while(again){
            std::cout<<"Press '+' to add increase the zone"<<std::endl;
            cin >> more;
            switch(more){
                case '+':
                    offset =MedusaBackend::getInstance()->getOffset(1, offset);
                    AView->showSetOfTets(MedusaBackend::getInstance()->gmdsToVtkIDs(offset));
                    break;
                default:
                    cin.clear();
                    again = false;
            }
        }

        MedusaBackend::getInstance()->unconstrainFrameTets(offset);
        AView->addTetSetActor();

    }else if(m_mode == 2) {
        MedusaBackend::getInstance()->removeSurfFromSelection();
        AView->removeLastSelection();
    }
}
/*-----*/
void MediatorGeomPolicy::removeItem(int AID, Textview *AView) {
    std::cout<<"Not implemented yet"<<std::endl;
}
/*-----*/
void MediatorGeomPolicy::generate(GraphicView *AView) {

    //m_mode = 1;

    if(m_mode == 0){
        AView->showPoints(MedusaBackend::getInstance()->wrongGeomPoint());
        //MedusaBackend::getInstance()->generateFrameField();
        int nb_sing_lines = MedusaBackend::getInstance()->getNbSingLines();
        for(int i = 0; i<nb_sing_lines; i++){
            AView->buildSingLine(i, MedusaBackend::getInstance()->gmdsToVtkIDs(MedusaBackend::getInstance()->pickSingLine(i)));
        }
        m_mode = 1;
    }
}

```

```

    AView->textMode(1);
} else if(m_mode == 1){
    AView->update();
    //Etape pour g rer le frame field apr s sa g n ration
    AView->hideSetOfTets();
    AView->removeSelection();

    MedusaBackend::getInstance()->generateFrameField();
    AView->resetSingLine();

    int nb_sing_lines = MedusaBackend::getInstance()->getNbSingLines();
    std::cout<<"nb lines "<<nb_sing_lines<<std::endl;
    for(int i = 0; i<nb_sing_lines; i++){
        AView->buildSingLine(i, MedusaBackend::getInstance()->gmdsToVtkIDs(MedusaBackend::getInstance()->pickSingLine(i)));
    }
    MedusaBackend::getInstance()->noCut();

    AView->showPoints(MedusaBackend::getInstance()->wrongGeomPoint());
    /*char more = '0';
    std::cout<<"Do you want to go to geometry cutting session ? (y/n)"<<std::endl;
    cin >> more;
    switch(more){
        case 'n':
            m_mode = 1;
            break;
        case 'y':
            m_mode = 2;
            break;
        default:
            cin.clear();

    }
    cin.clear();
    //m_mode = 2;
    AView->textMode(m_mode);*/
}
/*else if(m_mode == 2){
    //Generation de la coupe
    if(MedusaBackend::getInstance()->generateCut()==0) {
        AView->updateCut();
        m_mode = 3;
    }else{
        MedusaBackend::getInstance()->noCut();
        std::cout<<"ERROR: No surface selected"<<std::endl;
    }
}*/else{
}
}
}
/*-----*/
void MediatorGeomPolicy::generate(TextView *AView) {
    if(m_mode < 2){
        std::cout<<"Frame field generated"<<std::endl;
    } else if(m_mode == 2){
        //Generation de la coupe
        std::cout<<"Cut generated"<<std::endl;
    } else{
    }
}
/*-----*/
void MediatorGeomPolicy::undoGenerate(GraphicView *AView) {
    MedusaBackend::getInstance()->undoCut();
    AView->toggleBRep();
}
/*-----*/
void MediatorGeomPolicy::showAxis(GraphicView *AView, vtkIdType ACellID) {
    std::cout<<"Not implemented yet"<<std::endl;
}
/*-----*/
void MediatorGeomPolicy::surfaceMode(GraphicView *AView){
    std::cout<<"Not implemented yet"<<std::endl;
}
/*-----*/
void MediatorGeomPolicy::singularityGraphToggle(GraphicView *AView) {
    AView->toggleSingGraph();
}
/*-----*/
void MediatorGeomPolicy::opacity(GraphicView *AView) {
    if(m_mode == 0){

    } else if(m_mode == 1){
        AView->opacity();
    } else if(m_mode == 2){
        AView->toggleCutOpacity();
    }
}
/*-----*/
void MediatorGeomPolicy::color(GraphicView *AView) {
    AView->toggleBRep();
}
}

```

```

/*-----*/
#ifndef GMDS_MEDIATORGEOMPOLICY_H
#define GMDS_MEDIATORGEOMPOLICY_H
/*-----*/
#include <medusa/control/MediatorPolicyItf.h>
/*-----*/

namespace medusa {
    class MediatorGeomPolicy: public MediatorPolicyItf {

    public:
        MediatorGeomPolicy();
        virtual ~MediatorGeomPolicy();

        /*
         * Pick a geom surface
         */
        int selectVTKCell(GraphicView* AView,
                        const int ADim, const vtkIdType AID, int AAxis);
        int selectVTKCell(TextView* AView,
                        const int ADim, const vtkIdType AID);
        void help(TextView* AView);

        void remove(GraphicView *AView);

        /*
         * Will generate the frame field if there is no one, will generate the geom cut otherwise.
         */
        void generate(GraphicView *AView);
        void generate(TextView *AView);

        /*
         * Remove the last geom surface selected
         */
        void removeItem(int ASheetID, GraphicView *AView);
        void removeItem(int ASheetID, TextView *AView);

        //-----
        /*
         * Do nothing for now
         */
        void showAxis(GraphicView *AView, vtkIdType ACellID);

        void surfaceMode(GraphicView *AView);
        //-----

        /*
         * Undo the last "generate" action done
         */
        void undoGenerate(GraphicView *AView);

        void singularityGraphToggle(GraphicView *AView);

        /*
         * Calling this method switch the opacity of the cutted zone alternatively between transparent to hided to opaque
         */
        void opacity(GraphicView *AView);

        /*
         * toggle the geom surfaces colors
         */
        void color(GraphicView *AView);

    protected:

        /*
         * 0 = input mode : Frame field not generated
         * 1 = frame mode : Frame field generated, can do all frame operations
         * 2 = geom mode : can do all geom operation
         */
        int m_mode = 0;

    };
}

#endif //GMDS_MEDIATORGEOMPOLICY_H

```

```
/*-----*/
#ifndef GMDS_MEDIATORPOLICYITF_H
#define GMDS_MEDIATORPOLICYITF_H
/*-----*/
#include <vtkType.h>
/*-----*/
namespace medusa {
/*-----*/
class GraphicView;
class TextView;
/*-----*/
class MediatorPolicyItf {
public:
    virtual int selectVTKCell(GraphicView* AView,
                              const int ADim, const vtkIdType AID, int AAxis)=0;
    virtual int selectVTKCell(TextView* AView,
                              const int ADim, const vtkIdType AID)=0;

    virtual void help(TextView* AView) = 0;

    virtual void remove(GraphicView *AView)=0;

    virtual void removeItem(int AID, GraphicView *AView) = 0;
    virtual void removeItem(int AID, TextView *AView) = 0;

    virtual void generate(GraphicView *AView) = 0;
    virtual void generate(TextView *AView) = 0;
    virtual void undoGenerate(GraphicView *AView) = 0;

    virtual void showAxis(GraphicView *AView, vtkIdType ACellID) = 0;

    virtual void surfaceMode(GraphicView *AView) = 0;

    virtual void singularityGraphToggle(GraphicView *AView) = 0;

    virtual void opacity(GraphicView *AView) = 0;

    virtual void color(GraphicView *AView) = 0;
};
/*-----*/
}
/*-----*/
#endif //GMDS_MEDIATORPOLICYITF_H
/*-----*/
```

```

/*-----*/
#include <medusa/model/MedusaBackend.h>
#include <gmsh/io/VTKReader.h>
#include <gmsh/io/IGMeshIOService.h>

#include <gmsh/frame3d/FieldGenerator.h>
#include <gmsh/frame3d/OpenNLFieldSolverStrategy.h>
#include <gmsh/io/VTKWriter.h>

#include <utility>
/*-----*/
using namespace medusa;
using namespace gmsh;
MedusaBackend* MedusaBackend::m_instance = NULL;
/*-----*/
MedusaBackend* MedusaBackend::getInstance() {return m_instance;}
/*-----*/
void MedusaBackend::initialize()
{
    if(m_instance==NULL)
        m_instance = new MedusaBackend();
}
/*-----*/
MedusaBackend::MedusaBackend()
{}
/*-----*/
MedusaBackend::~MedusaBackend()
{
    for(auto g: m_grids){
        delete g;
    }
}
/*-----*/
std::vector<MedusaGrid*> MedusaBackend::grids() {
    return m_grids;
}
/*-----*/
void MedusaBackend::addObserver(medusa::MedusaObserver *AObs) {
    m_observer.push_back(AObs);
}
/*-----*/
void MedusaBackend::notify() {
    for(auto obs:m_observer){
        obs->update();
    }
}
/*-----*/
void MedusaBackend::loadVTK(const std::string &AFileName) {

    m_grids.push_back(new MedusaGrid(AFileName));
    m_grids[0]->name = "Tets";
    int nb_isolated_node = 0;
    int nb_nodes = m_grids[0]->getMesh()->getNbNodes();
    std::cout<<nb_nodes<<std::endl;

    for(auto n : m_grids[0]->getMesh()->nodes()){
        Node node = m_grids[0]->getMesh()->getNode(n);
        if(node.nbRegions() == 0){
            std::cout<<"isolated node"<<std::endl;
            nb_isolated_node++;
            m_grids[0]->getMesh()->deleteNode(n);
        }
    }
    int nb_nodes2 = m_grids[0]->getMesh()->getNbNodes();
    std::cout<<nb_nodes2<<std::endl;
    //startBlockingSession();
    initGeometry();

    IGMeshIOService ioService_m(m_grids[0]->getMesh());
    VTKWriter vtkWriter_m(&ioService_m);
    vtkWriter_m.setCellOptions(gmsh::N|gmsh::R);
    vtkWriter_m.setDataOptions(gmsh::N|gmsh::R);
    vtkWriter_m.write("/home/simon/Data/Results_debug/test.vtk");

    frameFieldInit(false);
    buildSingLines();
    startCuttingSession();
    BRep();
    notify();
}
/*-----*/
void MedusaBackend::loadVTK(const std::string &AFileName, bool FF, double AAngle, std::vector<int> ASurfs) {

    m_grids.push_back(new MedusaGrid(AFileName));
    m_grids[0]->name = "Tets";
    //startBlockingSession();
    initGeometry(AAngle);
    if(!ASurfs.empty()){
        for(auto s : ASurfs)

```

```

        unconstrainFrameTets(getTetsOfSurface(s));
    }
    frameFieldInit(FF);
    buildSingLines();
    startCuttingSession();
    BRep();
    notify();
}
/*-----*/
void MedusaBackend::modeBlocks() {
    if(m_grids.size() != 1) {
        std::cout<<"grid name = "<<m_grids[1]->name<<std::endl;
        m_grids.pop_back();
    }
    startBlockingSession();
    notify();
}
/*-----*/
void MedusaBackend::BRep() {
    MedusaGrid* BRep = new MedusaGrid(m_grids[0]->getMesh(), 1);
    BRep->name = "BRep";
    m_grids.push_back(BRep);
}
/*-----*/
std::vector<vtkIdType> MedusaBackend::createSurface(const int ADim, const vtkIdType &AVTKId, int &sheet_ID, int AAxis) {
    TCellID gmds_id = m_grids[0]->getGMDSCellID(ADim,AVTKId);

    MedusaGrid* surface_grid = new MedusaGrid();
    std::vector<TCellID> surf_ids = m_session->createSurface(gmds_id,AAxis,sheet_ID,surface_grid->getMesh());
    std::vector<vtkIdType> gmds_to_vtk;
    if(sheet_ID != -1) {
        for (auto id : surf_ids) {
            gmds_to_vtk.push_back(m_grids[0]->getVTKCellID(3, id));
        }
        surface_grid->updateGrid(2);
        m_grids.push_back(surface_grid);
    }
    return gmds_to_vtk;
}
/*-----*/
std::vector<vtkIdType> MedusaBackend::createBoundary(const int ADim, const vtkIdType &AVTKId, int &sheet_ID) {
    TCellID gmds_id = m_grids[0]->getGMDSCellID(ADim,AVTKId);

    MedusaGrid* surface_grid = new MedusaGrid();
    std::vector<TCellID> surf_ids = m_session->createBoundary(gmds_id,sheet_ID,surface_grid->getMesh());
    std::vector<vtkIdType> gmds_to_vtk;
    for(auto id : surf_ids){
        gmds_to_vtk.push_back(m_grids[0]->getVTKCellID(3, id));
    }
    surface_grid->updateGrid(2);
    m_grids.push_back(surface_grid);
    return gmds_to_vtk;
}
/*-----*/
void MedusaBackend::startBlockingSession() {
    MedusaGrid* block = new MedusaGrid();
    block->name = "blocks";
    block->getMesh()->changeModel(MeshModel(DIM3|R|F|E|N|R2N|F2N | E2N |
                                           R2F | F2R | F2E | E2R | E2F | R2E | N2R | N2F | N2E));
    m_grids.push_back(block);
    m_session = new db::DualBlockingSession(m_grids[0]->getMesh(), block->getMesh(),manager,linkerG_T);
}
/*-----*/
void MedusaBackend::startCuttingSession() {
    m_geom_cut = new graph::TopologyGraph(&manager,&linkerG_T,m_grids[0]->getMesh());
}
/*-----*/
int MedusaBackend::generateCut() {
    if(m_geom_cut->generateCut() == 0) {
        m_grids[0]->updateCutData();
        m_grids[1]->UpdateBRep();
        return 0;
    }
    return -1;
}
/*-----*/
void MedusaBackend::resetCut() {
    m_geom_cut->resetCut();
    m_grids[0]->updateCutData();
    m_grids[1]->UpdateBRep();
}
/*-----*/
void MedusaBackend::undoCut() {
    m_geom_cut->undoLastCut();
    m_grids[0]->updateCutData();
    m_grids[1]->UpdateBRep();
}
/*-----*/
void MedusaBackend::removeSurfFromSelection() {
    m_geom_cut->undoSelection();
}

```

```

}
/*-----*/
int MedusaBackend::pickSurface(TCellID AID, std::vector<vtkIdType> &AVtkIDs){
    std::vector<TCellID> ids;
    int surf_id = m_geom_cut->pickSurf(AID, ids);
    if(surf_id == -1) return -1;
    for(auto id : ids){
        AVtkIDs.push_back(m_grids[0]->getVTKCellID(3, id));
    }
    return surf_id;
}
/*-----*/
std::vector<TCellID> MedusaBackend::getTetsOfSurface(int AID){
    std::vector<TCellID> ids;
    m_geom_cut->pickSurf(AID, ids);
    return ids;
}
/*-----*/
void MedusaBackend::removeSheet(int ASheetID){
    m_session->deleteSheet(ASheetID);
}
/*-----*/
int MedusaBackend::generateDual(){
    int nbZones=0;
    m_session->refineSurfaceSheet();
    m_session->testLissageSheet();
    m_grids[0]->updateGrid(0);
    if(m_session->colorDual(nbZones)) {
        m_grids[0]->updateDualData();
    }else{
        nbZones = -2;
    }
    m_grids[0]->updateDualData();

    return nbZones;
}
/*-----*/
int MedusaBackend::reGenerateDual(){
    int nbZones = m_session->recolorDual();
    m_grids[0]->updateDualData();
    return nbZones;
}
/*-----*/
int MedusaBackend::correctDualRegions(){
    return m_session->removeWrongDualRegions();
}
/*-----*/
bool MedusaBackend::getFrameAxis(double ACoords[][3],vtkIdType ACellID){
    TCellID gmds_id = m_grids[0]->getGMDSCellID(3,ACellID);

    return m_session->getFrameAxis(ACoords, gmds_id);
}
/*-----*/
void MedusaBackend::generateBlocks(){
    m_session->createBlock();
    m_grids[1]->updateGrid(1);
}
/*-----*/
void MedusaBackend::resetDual(){
    m_session->hardResetDual();
}
/*-----*/
void MedusaBackend::resetBlock(){
    m_session->resetBlocks();
}
/*-----*/
std::vector<vtkIdType> MedusaBackend::getSingGraph(){
    std::vector<TCellID> gmds_graph = m_grids[0]->getSingularGraph();
    std::vector<vtkIdType> vtk_graph;
    for(auto g_id : gmds_graph){
        vtk_graph.push_back(m_grids[0]->getVTKCellID(3, g_id));
    }
    return vtk_graph;
}
/*-----*/
void MedusaBackend::initGeometry(double AAngle){
    manager.initAndLinkFrom3DMesh(m_grids[0]->getMesh(),&linkerG_T, AAngle);

    IGMeshIOService ioService_m(&manager.getMeshView());
    VTKWriter vtkWriter_m(&ioService_m);
    vtkWriter_m.setCellOptions(gmds::N|gmds::F);
    vtkWriter_m.setDataOptions(gmds::N|gmds::F);
    vtkWriter_m.write("/home/simon/Data/Results_debug/BRep.vtk");
}
/*-----*/
void MedusaBackend::frameFieldInit(bool FF){
    m_free_bnd = m_grids[0]->getMesh()->newVariable<int,GMDS_NODE>("free_bnd_nodes");

    if(!FF)
        generateFrameField();

    Variable<math::Chart>* vertex_chart = m_grids[0]->getMesh()->getOrCreateVariable<math::Chart, GMDS_NODE>("chart_field");

```



```

Variable<math::Vector3d>* var_X_field = m_grids[0]->getMesh()->getVariable<math::Vector3d, GMDS_NODE>("FF_X_POS");
Variable<math::Vector3d>* var_Y_field = m_grids[0]->getMesh()->getVariable<math::Vector3d, GMDS_NODE>("FF_Y_POS");
Variable<math::Vector3d>* var_Z_field = m_grids[0]->getMesh()->getVariable<math::Vector3d, GMDS_NODE>("FF_Z_POS");

for(auto n : m_grids[0]->getMesh()->nodes()){
    math::Vector3d x = var_X_field->value(n);
    math::Vector3d y = var_Y_field->value(n);
    math::Vector3d z = var_Z_field->value(n);
    math::Chart c(x,y,z);
    vertex_chart->set(n, c);
}
}

/*-----*/
void MedusaBackend::generateFrameField() {

    //Ici on build le frame field sur le mesh en input
    ParamsGlobal pg;
    ParamsFrameField pf;
    ParamsMark pm;

    frameFieldGenerateInit(&pg, &pf, &pm);

    OpenNLFieldSolverStrategy* solver = new OpenNLFieldSolverStrategy();
    FieldGenerator ffg(solver, m_grids[0]->getMesh(), pg, pf, pm);
    ffg.relaxBoundary(m_free_bnd);
    ffg.execute();

    buildSingLines();

    freeFrameFieldMarks(&pm);

    Variable<math::Chart>* vertex_chart = m_grids[0]->getMesh()->getOrCreateVariable<math::Chart, GMDS_NODE>("chart_field");

    Variable<math::Vector3d>* var_X_field = m_grids[0]->getMesh()->getVariable<math::Vector3d, GMDS_NODE>("FF_X_POS");
    Variable<math::Vector3d>* var_Y_field = m_grids[0]->getMesh()->getVariable<math::Vector3d, GMDS_NODE>("FF_Y_POS");
    Variable<math::Vector3d>* var_Z_field = m_grids[0]->getMesh()->getVariable<math::Vector3d, GMDS_NODE>("FF_Z_POS");

    for(auto n : m_grids[0]->getMesh()->nodes()){
        math::Vector3d x = var_X_field->value(n);
        math::Vector3d y = var_Y_field->value(n);
        math::Vector3d z = var_Z_field->value(n);
        math::Chart c(x,y,z);
        vertex_chart->set(n, c);
    }
    IGMeshIOService ioService_m(m_grids[0]->getMesh());
    VTKWriter vtkWriter_m(&ioService_m);
    vtkWriter_m.setCellOptions(gmnds::N|gmnds::R);
    vtkWriter_m.setDataOptions(gmnds::N|gmnds::R);
    vtkWriter_m.write("/home/simon/Data/Results_debug/result_FF.vtk");

    std::cout<<"\n\tFrame field generated"<<std::endl;
}

/*-----*/
void MedusaBackend::frameFieldGenerateInit(ParamsGlobal* AParamGlobal,
                                           ParamsFrameField* AParamFF,
                                           ParamsMark* AParamMark){

    AParamGlobal->algo_choice=ParamsGlobal::STABLE_HEX_GENERATION;
    AParamGlobal->start_from=ParamsGlobal::FF_GEN;
    AParamGlobal->stop_at=ParamsGlobal::FF_SMOOTH;
    AParamGlobal->with_debug_files=true;
    AParamGlobal->with_quad_bnd_constraint=false;

    AParamFF->solver_type=ParamsFrameField::OPENNL;
    AParamFF->with_cotangent_weights=false;
    AParamFF->with_smoothing=false;
    AParamFF->smoothing_algo=ParamsFrameField::RAY;
    AParamFF->smoothing_nb_iter=10;
    AParamFF->smoothing_epsilon=1e-6;
    AParamFF->with_mesh_adaptation=false;
    AParamFF->premeshing_sing_lines=false;

    AParamMark->mark_node_on_surf = m_grids[0]->getMesh()->newMark<Node>();
    AParamMark->mark_node_on_curv = m_grids[0]->getMesh()->newMark<Node>();
    AParamMark->mark_node_on_pnt = m_grids[0]->getMesh()->newMark<Node>();
    AParamMark->mark_node_isolated= m_grids[0]->getMesh()->newMark<Node>();

    AParamMark->mark_node_frame = m_grids[0]->getMesh()->newMark<Node>();
    AParamMark->mark_node_hard = m_grids[0]->getMesh()->newMark<Node>();

    AParamMark->mark_edge_on_surf = m_grids[0]->getMesh()->newMark<Edge>();
    AParamMark->mark_edge_on_curv = m_grids[0]->getMesh()->newMark<Edge>();

    AParamMark->mark_face_on_surf = m_grids[0]->getMesh()->newMark<Face>();

    BoundaryOperator boundaryOp(m_grids[0]->getMesh());

    if (!boundaryOp.isValid()) {
        std::cout << "Invalid model for boundary operations" << std::endl;
        throw GMDSException("Invalid model for boundary operations");
    }
}

```

```

//=====
// Mark boundary cells
boundaryOp.markCellOnGeometry(AParamMark->mark_face_on_surf,
                              AParamMark->mark_edge_on_surf,
                              AParamMark->mark_node_on_surf,
                              AParamMark->mark_edge_on_curv,
                              AParamMark->mark_node_on_curv,
                              AParamMark->mark_node_on_pnt,
                              AParamMark->mark_node_isolated);

//=====
// COLOR SURFACE AND CURVE NODES AND ASSIGN BND NORMALS

// Color names used to name the mesh variables are defined in
// the boundary operator class
boundaryOp.colorEdges(AParamMark->mark_edge_on_curv, AParamMark->mark_node_on_pnt);

boundaryOp.colorNodes(AParamMark->mark_node_on_pnt);

for (auto n_id : m_grids[0]->getMesh()->nodes()) {
    Node n = m_grids[0]->getMesh()->getNode(n_id);
    if (m_grids[0]->getMesh()->isMarked(n, AParamMark->mark_node_on_surf)) {
        math::Vector3d nv = boundaryOp.getOutputNormalOfABoundaryNode(n);
        m_bnd_normals[n.id()] = math::Vector3d(nv.X(), nv.Y(), nv.Z());
    }
}

// now we color nodes on curves and surfaces
Variable<int>* color_f = m_grids[0]->getMesh()->getVariable<int, GMDS_FACE>("BND_SURFACE_COLOR");
Variable<int>* color_c = m_grids[0]->getMesh()->getVariable<int, GMDS_EDGE>("BND_CURVE_COLOR");

Variable<int>* color_nf = 0;
try {
    color_nf = m_grids[0]->getMesh()->newVariable<int, GMDS_NODE>("BND_SURFACE_COLOR");
} catch (GMDSEException& e) {
    color_nf = m_grids[0]->getMesh()->getVariable<int, GMDS_NODE>("BND_SURFACE_COLOR");
}
Variable<int>* color_nc = 0;
try {
    color_nc = m_grids[0]->getMesh()->newVariable<int, GMDS_NODE>("BND_CURVE_COLOR");
} catch (GMDSEException& e) {
    color_nc = m_grids[0]->getMesh()->getVariable<int, GMDS_NODE>("BND_CURVE_COLOR");
}

for (auto f_id: m_grids[0]->getMesh()->faces()) {
    Face f = m_grids[0]->getMesh()->get<Face>(f_id);

    // only faces on surface are of interest
    if (!m_grids[0]->getMesh()->isMarked(f, AParamMark->mark_face_on_surf))
        continue;

    std::vector<Node> f_nodes = f.get<Node>();
    for (auto ni : f_nodes) {
        if (m_grids[0]->getMesh()->isMarked(ni, AParamMark->mark_node_on_surf) &&
            !m_grids[0]->getMesh()->isMarked(ni, AParamMark->mark_node_on_curv) &&
            !m_grids[0]->getMesh()->isMarked(ni, AParamMark->mark_node_on_pnt)) {
            (*color_nf)[ni.id()] = (*color_f)[f.id()];
        }
    }
}

for (auto e_id:m_grids[0]->getMesh()->edges()) {
    Edge e = m_grids[0]->getMesh()->get<Edge>(e_id);
    // only edges on surface are of interest
    if (!m_grids[0]->getMesh()->isMarked(e, AParamMark->mark_edge_on_curv))
        continue;

    std::vector<Node> e_nodes = e.get<Node>();
    for (auto ni : e_nodes) {
        if (m_grids[0]->getMesh()->isMarked(ni, AParamMark->mark_node_on_curv) &&
            !m_grids[0]->getMesh()->isMarked(ni, AParamMark->mark_node_on_pnt)) {
            (*color_nc)[ni.id()] = (*color_c)[e.id()];
        }
    }
}

Variable<double>* cot_w;
try {
    cot_w = m_grids[0]->getMesh()->getVariable<double, GMDS_EDGE>("cot_weight");
} catch (GMDSEException &e){
    cot_w = m_grids[0]->getMesh()->newVariable<double, GMDS_EDGE>("cot_weight");
}

// All weights equal to 1
for (auto e_id:m_grids[0]->getMesh()->edges()) {
    (*cot_w)[e_id] = 1;
}

}
/*-----*/
void MedusaBackend::freeFrameFieldMarks (ParamsMark* AParamsMark) {

```

```

m_grids[0]->getMesh()->unmarkAll<Node>(AParamsMark->mark_node_on_surf );
m_grids[0]->getMesh()->unmarkAll<Node>(AParamsMark->mark_node_on_curv );
m_grids[0]->getMesh()->unmarkAll<Node>(AParamsMark->mark_node_on_pnt );
m_grids[0]->getMesh()->unmarkAll<Node>(AParamsMark->mark_node_isolated);
m_grids[0]->getMesh()->unmarkAll<Node>(AParamsMark->mark_node_frame );
m_grids[0]->getMesh()->unmarkAll<Node>(AParamsMark->mark_node_hard );
m_grids[0]->getMesh()->unmarkAll<Edge>(AParamsMark->mark_edge_on_surf );
m_grids[0]->getMesh()->unmarkAll<Edge>(AParamsMark->mark_edge_on_curv );
m_grids[0]->getMesh()->unmarkAll<Face>(AParamsMark->mark_face_on_surf );

m_grids[0]->getMesh()->freeMark<Node>(AParamsMark->mark_node_on_surf );
m_grids[0]->getMesh()->freeMark<Node>(AParamsMark->mark_node_on_curv );
m_grids[0]->getMesh()->freeMark<Node>(AParamsMark->mark_node_on_pnt );
m_grids[0]->getMesh()->freeMark<Node>(AParamsMark->mark_node_isolated);
m_grids[0]->getMesh()->freeMark<Node>(AParamsMark->mark_node_frame );
m_grids[0]->getMesh()->freeMark<Node>(AParamsMark->mark_node_hard );
m_grids[0]->getMesh()->freeMark<Edge>(AParamsMark->mark_edge_on_surf );
m_grids[0]->getMesh()->freeMark<Edge>(AParamsMark->mark_edge_on_curv );
m_grids[0]->getMesh()->freeMark<Face>(AParamsMark->mark_face_on_surf );
}
/*-----*/
void MedusaBackend::buildSingLines() {
    m_sing_lines = m_grids[0]->getSingLines();
}
/*-----*/
std::vector<TCellID> MedusaBackend::pickSingLine(int ALineID) {
    return m_sing_lines[ALineID];
}
/*-----*/
int MedusaBackend::getNbSingLines() {
    return m_sing_lines.size();
}
/*-----*/
std::vector<vtkIdType> MedusaBackend::gmdsToVtkIDs(std::vector<TCellID> AIDs) {
    std::vector<vtkIdType> result;
    result.reserve(AIDs.size());

    for(auto id : AIDs) {
        result.push_back(m_grids[0]->getVTKCellID(3, id));
    }

    return result;
}
/*-----*/
std::vector<TCellID> MedusaBackend::vtkToGMDSEIDs(std::vector<vtkIdType> AIDs) {
    std::vector<TCellID> result;
    result.reserve(AIDs.size());

    for(auto id : AIDs) {
        result.push_back(m_grids[0]->getGMDSCellID(3, id));
    }

    return result;
}
/*-----*/
std::vector<TCellID> MedusaBackend::getOffset(int ADepth, std::vector<TCellID> AIDs) {
    std::vector<TCellID> result = grids()[0]->getOffset(ADepth, AIDs);
    return result;
}
/*-----*/
int MedusaBackend::unconstrainFrameTets(const std::vector<TCellID> AIDs) {
    for(auto t : AIDs) {
        Region r = grids()[0]->getMesh()->get<Region>(t);
        for(auto n : r.getID<Node>())
            m_free_bnd->set(n, 1);
    }

    return 0;
}
/*-----*/
bool MedusaBackend::checkOffsetBoundary(std::vector<TCellID> AIDs) {
}
/*-----*/
void MedusaBackend::noCut() {
    m_geom_cut->noCut();
}
/*-----*/
std::vector<TCellID> MedusaBackend::getWrongGeomPoint() {
    std::vector<int> wrong_points;

    std::vector<TCellID> gmds_node;

    Variable<math::Chart*> vertex_chart = m_grids[0]->getMesh()->getVariable<math::Chart, GMDS_NODE>("chart_field");

    std::vector<cad::GeomPoint*> points;
    manager.getPoints(points);

    for(auto p : points) {

```

```

for(auto s : p->surfaces()){

    std::vector<cad::GeomCurve*> curves_on_surface;
    for(auto c : p->curves()){
        if(std::find(c->surfaces().begin(),c->surfaces().end(), s) != c->surfaces().end()){
            curves_on_surface.push_back(c);
        }
    }

    // Les courbes geom incidentes au point sur la surface
    if(curves_on_surface.size() == 2) {
        cad::GeomCurve *c1 = curves_on_surface[0];
        cad::GeomCurve *c2 = curves_on_surface[1];

        Node n;
        Node n1;
        Node n2;

        math::Vector3d face_norm;

        for (auto mesh_node : m_grids[0]->getMesh()->nodes()) {
            if (linkerG_T.getGeomDim<Node>(mesh_node) == 1 && linkerG_T.getGeomId<Node>(mesh_node) == p->id()) {
                n = m_grids[0]->getMesh()->get<Node>(mesh_node);
            }
        }

        //POUR NOUVELLE METHODE
        Edge e1;
        Edge e2;

        for (auto e_n : n.get<Edge>()) {
            std::vector<Node> node_e = e_n.get<Node>();
            if (node_e[0].id() == n.id()) {
                if (linkerG_T.getGeomDim(node_e[1]) == 2) {
                    if (linkerG_T.getGeomId(node_e[1]) == c1->id()) {
                        n1 = node_e[1];
                        for (auto f : e_n.get<Face>()) {
                            if (linkerG_T.getGeomDim(f) == 3 && linkerG_T.getGeomId(f) == s->id()) {
                                face_norm = f.normal();
                            }
                        }
                    } else if (linkerG_T.getGeomId(node_e[1]) == c2->id()) {
                        n2 = node_e[1];
                        for (auto f : e_n.get<Face>()) {
                            if (linkerG_T.getGeomDim(f) == 3 && linkerG_T.getGeomId(f) == s->id()) {
                                face_norm = f.normal();
                            }
                        }
                    }
                }
            } else {
                if (linkerG_T.getGeomDim(node_e[0]) == 2) {
                    if (linkerG_T.getGeomId(node_e[0]) == c1->id()) {
                        n1 = node_e[0];
                        for (auto f : e_n.get<Face>()) {
                            if (linkerG_T.getGeomDim(f) == 3 && linkerG_T.getGeomId(f) == s->id()) {
                                face_norm = f.normal();
                            }
                        }
                    } else if (linkerG_T.getGeomId(node_e[0]) == c2->id()) {
                        n2 = node_e[0];
                        for (auto f : e_n.get<Face>()) {
                            if (linkerG_T.getGeomDim(f) == 3 && linkerG_T.getGeomId(f) == s->id()) {
                                face_norm = f.normal();
                            }
                        }
                    }
                }
            }
        }

        math::Chart chart = vertex_chart->value(n.id());
        math::Chart chart1 = vertex_chart->value(n1.id());
        math::Chart chart2 = vertex_chart->value(n2.id());

        double max = 0;
        int i_max = 0;
        for (int i = 0; i < 3; i++) {
            math::Vector3d vec = chart1[i];
            double product = vec.dot(face_norm);
            product = fabs(product);
            if (product > max) {
                max = product;
                i_max = i;
            }
        }

        //-----
        //TEST D'UNE NOUVELLE METHODE

```

```

//-----
std::vector<math::Vector3d> starting_vecs;
if (i_max == 0) {
    starting_vecs.push_back(chart1[1]);
    starting_vecs.push_back(chart1[2]);
} else if (i_max == 1) {
    starting_vecs.push_back(chart1[0]);
    starting_vecs.push_back(chart1[2]);
} else {
    starting_vecs.push_back(chart1[0]);
    starting_vecs.push_back(chart1[1]);
}

bool wrong_point = false;
for (auto starting_vec : starting_vecs) {

    //On propage cette direction correspondante entre n2 et n0
    math::Vector3d closest_vec_1_to_0;
    max = 0;
    for (int i = 0; i < 3; i++) {
        math::Vector3d vec = chart[i];
        double product = vec.dot(starting_vec);
        product = fabs(product);
        if (product > max) {
            max = product;
            closest_vec_1_to_0 = vec;
        }
    }
    if (closest_vec_1_to_0.dot(starting_vec) < 0)
        closest_vec_1_to_0 = -1 * closest_vec_1_to_0;

    //std::cout<<"starting vec : "<<starting_vec<<std::endl;
    //std::cout<<"closest_vec_1_0 : "<<closest_vec_1_to_0<<std::endl;

    math::Vector3d closest_vec_0_to_2;
    max = 0;
    for (int i = 0; i < 3; i++) {
        math::Vector3d vec = chart2[i];
        double product = vec.dot(closest_vec_1_to_0);
        product = fabs(product);
        if (product > max) {
            max = product;
            closest_vec_0_to_2 = vec;
        }
    }
    if (closest_vec_0_to_2.dot(closest_vec_1_to_0) < 0)
        closest_vec_0_to_2 = -1 * closest_vec_0_to_2;

    //std::cout<<"closest_vec_1_0 : "<<closest_vec_1_to_0<<std::endl;
    //std::cout<<"closest_vec_0_2 : "<<closest_vec_0_to_2<<std::endl;

    //On cherche la direction correspondante au vector 0 du frame de n1 pour n2
    math::Vector3d closest_vec_2_to_1;
    max = 0;
    for (int i = 0; i < 3; i++) {
        math::Vector3d vec = chart1[i];
        double product = vec.dot(closest_vec_0_to_2);
        product = fabs(product);
        if (product > max) {
            max = product;
            closest_vec_2_to_1 = vec;
        }
    }
    if (closest_vec_2_to_1.dot(closest_vec_0_to_2) < 0)
        closest_vec_2_to_1 = -1 * closest_vec_2_to_1;

    if (starting_vec != closest_vec_2_to_1) {
        std::cout<<"starting vec : "<<starting_vec<<std::endl;
        std::cout<<"closest_vec : "<<closest_vec_2_to_1<<std::endl;
        //gmds_node.push_back(n.id());
        wrong_point = true;
        std::cout << "Wrong point "<<p->id()<<" on surf "<<s->id()<< std::endl;
    }

    /*
    //On cherche la direction correspondante au vector 0 du frame de n1 pour n2
    math::Vector3d closest_vec_1_to_2;
    max = 0;
    for (int i = 0; i < 3; i++) {
        math::Vector3d vec = chart2[i];
        double product = vec.dot(starting_vec);
        product = fabs(product);
        if (product > max) {
            max = product;
            closest_vec_1_to_2 = vec;
        }
    }
    if (closest_vec_1_to_2.dot(starting_vec) < 0)

```

```

        closest_vec_1_to_2 = -1 * closest_vec_1_to_2;

        std::cout << closest_vec_1_to_2 << std::endl;

        //On propage cette direction correspondante entre n2 et n0
        math::Vector3d closest_vec_2_to_0;
        max = 0;
        for (int i = 0; i < 3; i++) {
            math::Vector3d vec = chart[i];
            double product = vec.dot(closest_vec_1_to_2);
            product = fabs(product);
            if (product > max) {
                max = product;
                closest_vec_2_to_0 = vec;
            }
        }
        if (closest_vec_2_to_0.dot(closest_vec_1_to_2) < 0)
            closest_vec_2_to_0 = -1 * closest_vec_2_to_0;

        std::cout << closest_vec_2_to_0 << std::endl;

        math::Vector3d closest_vec_0_to_1;
        max = 0;
        for (int i = 0; i < 3; i++) {
            math::Vector3d vec = chart1[i];
            double product = vec.dot(closest_vec_2_to_0);
            product = fabs(product);
            if (product > max) {
                max = product;
                closest_vec_0_to_1 = vec;
            }
        }
        if (closest_vec_0_to_1.dot(closest_vec_2_to_0) < 0)
            closest_vec_0_to_1 = -1 * closest_vec_0_to_1;

        std::cout << closest_vec_0_to_1 << std::endl;

        if (starting_vec != closest_vec_0_to_1) {
            //gmds_node.push_back(n.id());
            wrong_point = true;
            std::cout << "Wrong point" << std::endl;
        }

        /*if(s->angle(p)<(1.5708/2)){
            wrong_points.push_back(p->id());
            break;
        }*/

    }
    if (wrong_point) gmds_node.push_back(n.id());
}

}

/*for(auto n : m_grids[0]->getMesh()->nodes()){
    if(linkerG_T.getGeomDim<Node>(n) == 1) {
        if (std::find(wrong_points.begin(), wrong_points.end(), linkerG_T.getGeomId<Node>(n)) !=
            wrong_points.end()) {
            gmds_node.push_back(n);
        }
    }
}*/

return gmds_node;
}

/*-----*/
std::vector<vtkIdType> MedusaBackend::getVtkNode(std::vector<TCellID> AIDs) {
    std::vector<vtkIdType> vtk_ids;
    vtk_ids.reserve(AIDs.size());
    for(auto id : AIDs) {
        vtk_ids.push_back(m_grids[0]->getVTKCellID(0, id));
    }
    return vtk_ids;
}

/*-----*/
std::vector<std::vector<double>> MedusaBackend::wrongGeomPoint() {

    std::vector<std::vector<double>> result;

    std::vector<TCellID> points = getWrongGeomPoint();

    for(auto p : points){
        std::vector<double> tab;
        Node n = m_grids[0]->getMesh()->get<Node>(p);
        tab.push_back(n.getPoint().X());
        tab.push_back(n.getPoint().Y());
        tab.push_back(n.getPoint().Z());
        result.push_back(tab);
    }

    return result;
}

```

```

/*-----*/
#ifndef GMDs_MEDUSABACKEND_H
#define GMDs_MEDUSABACKEND_H
/*-----*/
#include <vector>
#include <string>
/*-----*/
#include <medusa/view/MedusaObserver.h>
#include <medusa/model/MedusaGrid.h>
#include <gmds/dualBlocking/DualBlockingSession.h>
/*-----*/
# include <gmds/frame3d/Params.h>
# include <gmds/graph/TopologyGraph.h>
# include <gmds/igalgo/BoundaryOperator.h>
# include <gmds/smoothy/LaplacianSmoother3C.h>
/*-----*/

namespace medusa{
    class MedusaBackend{
    public:
        /**
         * @brief initialize the unique instance of this class. It is mandatory
         * to call it since the getInstance() method does not test the
         * existence of this class instance.
         */
        static void initialize();

        /**
         * @brief returns the unique instance of MedusaBackend
         */
        static MedusaBackend* getInstance();

        virtual ~MedusaBackend();
        void loadVTK(const std::string& AFileName);
        void loadVTK(const std::string &AFileName, bool FF, double AAngle, std::vector<int> ASurfs);
        void addObserver(MedusaObserver* AObs);
        std::vector<MedusaGrid*> grids();

        std::vector<vtkIdType> gmdsToVtkIDs(std::vector<gmds::TCellID> AIDs);
        std::vector<gmds::TCellID> vtkToGMDsIDs(std::vector<vtkIdType> AIDs);
        std::vector<gmds::TCellID> getOffset(int ADepth, std::vector<gmds::TCellID> AIDs);

        void modeBlocks();

        //Frame operation
        std::vector<vtkIdType> getSingGraph();

        int getNbSingLines();

        bool getFrameAxis(double ACoords[][3],vtkIdType ACellID);
        void frameFieldInit(bool FF);
        void generateFrameField();

        std::vector<gmds::TCellID> pickSingLine(int ALineID);
        int unconstrainFrameTets(std::vector<gmds::TCellID> AIDs);

        //Dual operations
        std::vector<vtkIdType> createSurface(int ADim, const vtkIdType &AVTKId, int &ASheetID, int AAxis);
        std::vector<vtkIdType> createBoundary(int ADim, const vtkIdType &AVTKId, int &sheet_ID);
        void removeSheet(int ASheetID);

        int generateDual();
        void resetDual();

        void generateBlocks();
        void resetBlock();

        int reGenerateDual();
        int correctDualRegions();

        //Geom operations
        int pickSurface(gmds::TCellID AID, std::vector<vtkIdType>& AVtkIDs);
        void removeSurfFromSelection();

        int generateCut();

        void resetCut();

        void undoCut();

        void noCut();

        std::vector<std::vector<double>> wrongGeomPoint();

        std::vector<gmds::TCellID> getTetsOfSurface(int AID);

    private:
        MedusaBackend();
        void notify();
        void initGeometry(const double AAngle=0.72);
    };
}

```

```

    void startBlockingSession();
    void startCuttingSession();
    void BRep();

    void buildSingLines();

    bool checkOffsetBoundary(std::vector<gmds::TCellID> AIDs);

    void frameFieldGenerateInit(gmds::ParamsGlobal *AParamGlobal, gmds::ParamsFrameField *AParamFF, gmds::ParamsMark *AParamMar
k);
    void freeFrameFieldMarks(gmds::ParamsMark *AParamsMark);

private:
    static MedusaBackend* m_instance;
    std::vector<MedusaObserver*> m_observer;
    std::vector<MedusaGrid*> m_grids;
    db::DualBlockingSession* m_session = nullptr;

    gmds::graph::TopologyGraph* m_geom_cut = nullptr;

    gmds::cad::FACManager manager;
    gmds::cad::GeomMeshLinker linkerG_T;

    std::map<int, std::vector<gmds::TCellID>> m_sing_lines;
    gmds::Variable<int>* m_free_bnd;

    std::vector<gmds::TCellID> getWrongGeomPoint();

    std::vector<vtkIdType> getVtkNode(std::vector<gmds::TCellID> AIDs);

};

/*-----*/
#endif //GMDS_MEDUSABACKEND_H
/*-----*/

```



```
//  
// Created by calderans on 28/01/20.  
//  
  
#include "medusa/control/MedusaCommandGenerate.h"  
  
using namespace medusa;  
  
MedusaCommandGenerate::MedusaCommandGenerate(MediatorPolicyItf* APolicy, GraphicView* AGraphicView, TextView* ATextView):MedusaCom  
mand(APolicy,AGraphicView,ATextView) {  
  
}  
/*-----*/  
  
MedusaCommand::State MedusaCommandGenerate::execute() {  
  
    m_policy->generate(m_text_collaborator);  
    m_policy->generate(m_graphic_collaborator);  
    return SUCCES;  
}  
/*-----*/  
  
MedusaCommand::State MedusaCommandGenerate::undo() {  
    std::cout<<"Undo generate"<<std::endl;  
    m_policy->undoGenerate(m_graphic_collaborator);  
}  
/*-----*/
```

```
//  
// Created by calderans on 28/01/20.  
//  
  
#ifndef GMS_MEDUSACOMMANDGENERATE_H  
#define GMS_MEDUSACOMMANDGENERATE_H  
  
#include "MedusaCommand.h"  
  
namespace medusa{  
    class MedusaCommandGenerate: public MedusaCommand{  
  
    public:  
        MedusaCommandGenerate(MediatorPolicyItf* APolicy, GraphicView* AGraphicView, TextView* ATextView);  
        State execute();  
        State undo();  
  
    protected:  
  
        };  
}  
  
#endif //GMS_MEDUSACOMMANDGENERATE_H
```

```
//
// Created by calderans on 28/01/20.
//

#ifndef GMD5_MEDUSACOMMAND_H
#define GMD5_MEDUSACOMMAND_H

#include <medusa/view/GraphicView.h>
#include <medusa/view/TextView.h>
#include "MediatorPolicyItf.h"

namespace medusa{
    class MedusaCommand{
    public:
        enum State {SUCCES,FAILURE};
    public:
        MedusaCommand(MediatorPolicyItf* APolicy, GraphicView* AGraphicView, TextView* ATextView){
            m_policy = APolicy;
            m_graphic_collaborator = AGraphicView;
            m_text_collaborator = ATextView;
        }
        virtual ~MedusaCommand(){};
        virtual State execute() = 0;
        virtual State undo() = 0;

    protected:
        MediatorPolicyItf* m_policy;
        GraphicView* m_graphic_collaborator;
        TextView* m_text_collaborator;
    };
}

#endif //GMD5_MEDUSACOMMAND_H
```

```
//  
// Created by calderans on 28/01/20.  
//  
  
#include "medusa/control/MedusaCommandPick.h"  
  
using namespace medusa;  
  
MedusaCommandPick::MedusaCommandPick(MediatorPolicyItf* APolicy, GraphicView* AGraphicView, TextView* ATextView, int ADIM, vtkIdType ACellID):MedusaCommand(APolicy, AGraphicView, ATextView) {  
  
    m_DIM = ADIM;  
    m_CellID = ACellID;  
}  
/*-----*/  
  
MedusaCommand::State MedusaCommandPick::execute() {  
    //m_policy->showAxis(m_graphic_collaborator, m_CellID);  
  
    int axis = 0;  
    //axis = m_policy->selectVTKCell(m_text_collaborator, m_DIM, m_CellID);  
  
    //if(axis != 0) {  
        //We pick on the grid[0] which is the tet mesh  
        m_surfID = m_policy->selectVTKCell(m_graphic_collaborator, m_DIM, m_CellID, axis);  
  
        if(m_surfID != -1) {  
            std::cout<<"SUCCES: [pick]"<<std::endl;  
            return SUCCES;  
        }  
    }  
    return FAILURE;  
}  
/*-----*/  
  
MedusaCommand::State MedusaCommandPick::undo() {  
  
    m_policy->removeItem(m_surfID, m_graphic_collaborator);  
    m_policy->removeItem(m_surfID, m_text_collaborator);  
}  
/*-----*/
```

```
//  
// Created by calderans on 28/01/20.  
//  
  
#ifndef GMDS_MEDUSACOMMANDPICK_H  
#define GMDS_MEDUSACOMMANDPICK_H  
  
#include "MedusaCommand.h"  
  
namespace medusa{  
    class MedusaCommandPick: public MedusaCommand{  
  
    public:  
        MedusaCommandPick(MediatorPolicyItf* APolicy, GraphicView* AGraphicView, TextView* ATextView, int ADIM, vtkIdType ACellID);  
        State execute();  
        State undo();  
  
    protected:  
        int m_surfID;  
        int m_DIM;  
        vtkIdType m_CellID;  
    };  
}  
  
#endif //GMDS_MEDUSACOMMANDPICK_H
```

```
/*-----*/
#include <medusa/Medusa.h>
#include <medusa/model/MedusaBackend.h>
#include <medusa/view/GraphicView.h>
#include <medusa/control/MediatorExplorerPolicy.h>
#include <medusa/control/MediatorDubloPolicy.h>
#include <medusa/control/MediatorGeomPolicy.h>
/*-----*/
#include <vtkDataSetMapper.h>
#include <vtkRenderWindow.h>
#include <vtkRenderWindowInteractor.h>
#include <vtkCamera.h>
#include <vtkActor.h>
#include <vtkRenderer.h>
#include <vtkHexahedron.h>
#include <vtkProperty.h>
#include <vtkCamera.h>
/*-----*/
using namespace medusa;
using namespace gmds;
/*-----*/
Medusa::Medusa(const Medusa::Mode AMode) {
    init(AMode);
}
/*-----*/
Medusa::~Medusa() {
    for(auto v: m_gviews){
        if(v!=NULL)
            delete v;
    }
    for(auto v: m_tvviews){
        if(v!=NULL)
            delete v;
    }
}
/*-----*/
void Medusa::setMode(const Medusa::Mode AMode) {

    MediatorPolicyItf* policy=NULL;
    if(AMode==Medusa::DUAL_BLOCKING)
        policy = new MediatorDubloPolicy();
    else if(AMode==Medusa::EXPLORATION)
        policy = new MediatorExplorerPolicy();
    else if(AMode==Medusa::GEOMETRY)
        policy = new MediatorGeomPolicy();
    else
        throw GMDSException("Medusa Error: unknown mode!");

    MediatorControl::getInstance()->setPolicy(policy);
}
/*-----*/
void Medusa::launch() {
    if(m_last_gview==NULL)
        throw GMDSException("Medusa requires at least one graphic view to be launched!");
    m_last_gview->start();
}
/*-----*/
void Medusa::load(const std::string &AFileName) {
    //MedusaBackend::getInstance()->loadVTK(AFileName);
    MedusaBackend::getInstance()->loadVTK(AFileName);
}
/*-----*/
void Medusa::init(const Medusa::Mode AMode) {
    //Important, otherwise the MedusaBackend instance is not instanciated!!!
    MedusaBackend::initialize();
    MediatorControl::initialize();
    setMode(AMode);
    MedusaBackend::getInstance()->addObserver(MediatorControl::getInstance());
}
/*-----*/
GraphicView* Medusa::newGraphicView(const std::string& AName,
                                    GraphicView::ViewType AType){
    GraphicView* v = new GraphicView(AName, AType);
    m_gviews.push_back(v);
    MedusaBackend::getInstance()->addObserver(v);
    MediatorControl::getInstance()->addCollaborator(v);
    m_last_gview=v;
    return v;
}
/*-----*/
TextView* Medusa::newTextView() {
    TextView* v = new TextView();
    m_tvviews.push_back(v);
    MedusaBackend::getInstance()->addObserver(v);
    MediatorControl::getInstance()->addCollaborator(v);
    return v;
}
/*-----*/
```

```

/*-----*/
#include "medusa/model/MedusaGrid.h"
/*-----*/
#include <gmsh/io/IGMeshIOService.h>
#include <gmsh/io/VTKReader.h>
/*-----*/
#include <vtkCellArray.h>
#include <vtkPointData.h>
#include <vtkCellData.h>
#include <vtkUnstructuredGrid.h>
#include <vtkHexahedron.h>
/*-----*/
#include <map>
#include <gmsh/ig/MeshDoctor.h>
/*-----*/
using namespace medusa;
using namespace gmsh;

MedusaGrid::MedusaGrid()
: m_gmsh_mesh(MeshModel(DIM3|R|F|E|N|R2N|R2F|F2N|E2N|R2E|F2E)) {

    m_vtk_grid= vtkUnstructuredGrid::New();
    for(auto i=0;i<4;i++){
        m_vtk_to_gmsh[i]= vtkIdTypeArray::New();
        m_vtk_to_gmsh[i]->SetName("GMSH_ID");
    }
    /*vtkPoints *pts = vtkPoints::New();

    pts->SetNumberOfPoints(8);
    pts->InsertPoint(0, 0, 0, 0);
    pts->InsertPoint(1, 1, 0, 0);
    pts->InsertPoint(2, 1, 1, 0);
    pts->InsertPoint(3, 0, 1, 0);
    pts->InsertPoint(4, 0, 0, 1);
    pts->InsertPoint(5, 1, 0, 1);
    pts->InsertPoint(6, 1, 1, 1);
    pts->InsertPoint(7, 0, 1, 1);
    vtkHexahedron* aHexahedron = vtkHexahedron::New();
    aHexahedron->GetPointIds()->SetId(0, 0);
    aHexahedron->GetPointIds()->SetId(1, 1);
    aHexahedron->GetPointIds()->SetId(2, 2);
    aHexahedron->GetPointIds()->SetId(3, 3);
    aHexahedron->GetPointIds()->SetId(4, 4);
    aHexahedron->GetPointIds()->SetId(5, 5);
    aHexahedron->GetPointIds()->SetId(6, 6);
    aHexahedron->GetPointIds()->SetId(7, 7);
    m_vtk_grid->Allocate(1, 1);
    m_vtk_grid->InsertNextCell(aHexahedron->GetCellType(),
                             aHexahedron->GetPointIds());
    m_vtk_grid->SetPoints(pts);*/
}
/*-----*/
MedusaGrid::MedusaGrid(const std::string& AFileName)
: m_gmsh_mesh(MeshModel(DIM3|R|F|E|N|R2N|F2N|E2N|
                        R2F|F2R|F2E|E2R|E2F|R2E|N2R|N2F|N2E)) {

    m_vtk_grid= vtkUnstructuredGrid::New();
    //=====
    IGMeshIOService ioService(&m_gmsh_mesh);
    gmsh::VTKReader vtkReader(&ioService);
    vtkReader.setCellOptions(gmsh::N|gmsh::R);
    vtkReader.setDataOptions(gmsh::N|gmsh::R, false);
    vtkReader.read(AFileName);

    MeshDoctor doc(&m_gmsh_mesh);
    doc.buildFacesAndR2F();
    doc.buildEdgesAndX2E();
    doc.updateUpwardConnectivity();

    auto nbPoints = m_gmsh_mesh.getNbNodes();
    auto nbCells = m_gmsh_mesh.getNbRegions();

    for(auto i=0;i<4;i++){
        m_vtk_to_gmsh[i]= vtkIdTypeArray::New();
        m_vtk_to_gmsh[i]->SetName("GMSH_ID");
    }

    //===== POINTS CREATION =====
    vtkPoints* grid_points = vtkPoints::New();
    grid_points->SetNumberOfPoints(nbPoints);
    grid_points->SetDataTypeToDouble();
    m_vtk_to_gmsh[0]->SetNumberOfValues(nbPoints);

    //map to store the correspondence from a gmsh node id to a vtk point id

    std::map<TCellID, int> map_node_ids;
    int vtk_node_id=0;
    for(auto gmsh_node_id:m_gmsh_mesh.nodes()){
        //id mapping
        map_node_ids[gmsh_node_id]=vtk_node_id;
        //point definition

```

```

    math::Point loc = m_gmds_mesh.get<Node>(gmds_node_id).getPoint();
    grid_points->InsertPoint(vtk_node_id,loc.X(),loc.Y(),loc.Z());
    m_vtk_to_gmds[0]->InsertValue(vtk_node_id,gmds_node_id);
    vtk_node_id++;
}

m_vtk_grid->SetPoints(grid_points);
grid_points->Delete();

m_vtk_grid->GetPointData()->AddArray(m_vtk_to_gmds[0]);

//===== CELLS CREATION =====
m_vtk_grid->Allocate(nbCells);

m_vtk_to_gmds[3]->SetNumberOfValues(nbCells);

int vtk_cell_id=0;
for(auto gmds_cell_id:m_gmds_mesh.regions())
{
    Region r = m_gmds_mesh.get<Region>(gmds_cell_id);
    int vtk_type=0;
    switch (r.type()){
        case GMDs_HEX:
            vtk_type = VTK_HEXAHEDRON;
            break;
        case GMDs_TETRA:
            vtk_type = VTK_TETRA;
            break;
        case GMDs_PRISM3:
            vtk_type = VTK_WEDGE;
            break;
        case GMDs_PYRAMID:
            vtk_type = VTK_PYRAMID;
            break;
        default:
            throw GMDSException("Input GMDs mesh contains cell not handled by VTK");
    }

    std::vector<TCellID> node_ids = r.getIds<Node>();

    vtkIdList* idList = vtkIdList::New();
    idList->SetNumberOfIds(node_ids.size());
    int i_vtk_node = 0;
    for(auto i_node:node_ids){
        idList->SetId(i_vtk_node++, map_node_ids[i_node]);
    }
    m_vtk_to_gmds[3]->InsertValue(vtk_cell_id,gmds_cell_id);
    m_gmds_to_vtk[3].emplace(gmds_cell_id,vtk_cell_id);

    m_vtk_grid->InsertNextCell(vtk_type,idList);

    vtk_cell_id++;
}

m_vtk_grid->GetCellData()->AddArray(m_vtk_to_gmds[3]);

m_vtk_blocks = vtkSmartPointer<vtkIntArray>::New();
m_vtk_blocks->SetName("blocks");
m_vtk_blocks->SetNumberOfValues(m_gmds_mesh.getNbRegions());

for(vtkIdType i = 0; i < m_vtk_grid->GetNumberOfCells(); i++){
    m_vtk_blocks->InsertValue(i,0);
}

m_vtk_grid->GetCellData()->AddArray(m_vtk_blocks);

m_vtk_cut = vtkIntArray::New();
m_vtk_cut->SetName("cut");
m_vtk_cut->SetNumberOfValues(m_gmds_mesh.getNbRegions());

for(vtkIdType i = 0; i < m_vtk_grid->GetNumberOfCells(); i++){
    m_vtk_cut->InsertValue(i,1);
}

m_vtk_grid->GetCellData()->AddArray(m_vtk_cut);
}
/*-----*/
void MedusaGrid::updateCutData() {
    gmds::Variable<int>* cut;
    if(m_mode == 0) {
        cut = m_gmds_mesh.getVariable<int>(GMDs_REGION>("CUT_tet"));
    } else if(m_mode == 1){
        cut = m_gmds_mesh.getVariable<int>(GMDs_FACE>("CUT_face"));
    }

    for(vtkIdType i = 0; i < m_vtk_grid->GetNumberOfCells(); i++){
        TCellID gmds_id = getGMDSCellID(3,i);
        int value = cut->value(gmds_id);
        m_vtk_cut->SetValue(i,value);
    }
}

```



```

    }
}
/*-----*/
MedusaGrid::MedusaGrid(const Mesh* AMesh, int AMode):m_gmds_mesh(*AMesh) {

    /*
     * AMode =
     * 0 -> normal mode, building cells
     * 1 -> BRep mode, building only surf faces
     */

    m_mode = AMode;
    int nbPoints, nbCells;

    MeshDoctor doc(&m_gmds_mesh);
    //doc.buildFacesAndR2F();
    //doc.buildEdgesAndX2E();
    doc.updateUpwardConnectivity();

    m_vtk_grid= vtkUnstructuredGrid::New();

    for (auto i = 0; i < 4; i++) {
        m_vtk_to_gmds[i] = vtkIdTypeArray::New();
        m_vtk_to_gmds[i]->SetName("GMDs_ID");
    }

    if (m_mode == 0) {

        nbPoints = m_gmds_mesh.getNbNodes();
        nbCells = m_gmds_mesh.getNbRegions();

        //===== POINTS CREATION =====
        vtkPoints *grid_points = vtkPoints::New();
        grid_points->SetNumberOfPoints(nbPoints);
        grid_points->SetDataTypeToDouble();
        m_vtk_to_gmds[0]->SetNumberOfValues(nbPoints);

        //map to store the correspondence from a gmds node id to a vtk point id

        std::map<TCellID, int> map_node_ids;
        int vtk_node_id = 0;
        for (auto gmds_node_id:m_gmds_mesh.nodes()) {
            //id mapping
            map_node_ids[gmds_node_id] = vtk_node_id;
            //point definition
            math::Point loc = m_gmds_mesh.get<Node>(gmds_node_id).getPoint();
            grid_points->InsertPoint(vtk_node_id, loc.X(), loc.Y(), loc.Z());
            m_vtk_to_gmds[0]->InsertValue(vtk_node_id, gmds_node_id);
            vtk_node_id++;
        }

        m_vtk_grid->SetPoints(grid_points);
        grid_points->Delete();

        m_vtk_grid->GetPointData()->AddArray(m_vtk_to_gmds[0]);

        //===== CELLS CREATION =====
        m_vtk_grid->Allocate(nbCells);

        m_vtk_to_gmds[3]->SetNumberOfValues(nbCells);

        int vtk_cell_id = 0;
        for (auto gmds_cell_id:m_gmds_mesh.regions()) {
            Region r = m_gmds_mesh.get<Region>(gmds_cell_id);
            int vtk_type = 0;
            switch (r.type()) {
                case GMDs_HEX:
                    vtk_type = VTK_HEXAHEDRON;
                    break;
                case GMDs_TETRA:
                    vtk_type = VTK_TETRA;
                    break;
                case GMDs_PRISM3:
                    vtk_type = VTK_WEDGE;
                    break;
                case GMDs_PYRAMID:
                    vtk_type = VTK_PYRAMID;
                    break;
                default:
                    throw GMDSException("Input GMDs mesh contains cell not handled by VTK");
            }

            std::vector<TCellID> node_ids = r.getIds<Node>();

            vtkIdList *idList = vtkIdList::New();
            idList->SetNumberOfIds(node_ids.size());
            int i_vtk_node = 0;
            for (auto i_node:node_ids) {
                idList->SetId(i_vtk_node++, map_node_ids[i_node]);
            }
        }
    }
}

```

```

m_vtk_to_gmds[3]->InsertValue(vtk_cell_id, gmds_cell_id);
m_gmds_to_vtk[3].emplace(gmds_cell_id, vtk_cell_id);

m_vtk_grid->InsertNextCell(vtk_type, idList);

    vtk_cell_id++;
}

m_vtk_grid->GetCellData()->AddArray(m_vtk_to_gmds[3]);
} else if (m_mode == 1) {
    gmds::Variable<int>* BND_Color = m_gmds_mesh.getVariable<int, GMDS_FACE>("BND_SURFACE_COLOR");

    std::set<TCellID> nodes_set;
    std::vector<TCellID> BND_faces;

    for(auto f : m_gmds_mesh.faces()){
        if(BND_Color->value(f) != 0){
            Face face = m_gmds_mesh.get<Face>(f);
            BND_faces.push_back(f);
            for(auto n : face.getIds<Node>()){
                nodes_set.insert(n);
            }
        }
    }

    nbPoints = nodes_set.size();
    nbCells = BND_faces.size();

    //===== POINTS CREATION =====
    vtkPoints* grid_points = vtkPoints::New();
    grid_points->SetNumberOfPoints(nbPoints);
    grid_points->SetDataTypeToDouble();
    m_vtk_to_gmds[0]->SetNumberOfValues(nbPoints);

    std::map<TCellID, int> map_node_ids;
    int vtk_node_id=0;
    for(auto gmds_node_id:nodes_set){
        //id mapping
        map_node_ids[gmds_node_id]=vtk_node_id;
        //point definition
        math::Point loc = m_gmds_mesh.get<Node>(gmds_node_id).getPoint();
        grid_points->InsertPoint(vtk_node_id,loc.X(),loc.Y(),loc.Z());
        m_vtk_to_gmds[0]->InsertValue(vtk_node_id,gmds_node_id);
        vtk_node_id++;
    }
    m_vtk_grid->SetPoints(grid_points);
    grid_points->Delete();

    m_vtk_grid->GetPointData()->AddArray(m_vtk_to_gmds[0]);

    //===== FACES CREATION =====

    m_vtk_to_gmds[2]->SetNumberOfValues(nbCells);

    int vtk_face_id = 0;
    for (auto gmds_face_id:BND_faces) {
        Face f = m_gmds_mesh.get<Face>(gmds_face_id);
        std::vector<TCellID> node_ids = f.getIds<Node>();

        vtkIdList *idList = vtkIdList::New();
        idList->SetNumberOfIds(node_ids.size());
        int i_vtk_node = 0;
        for (auto i_node:node_ids) {
            idList->SetId(i_vtk_node++, map_node_ids[i_node]);
        }
        m_vtk_to_gmds[2]->InsertValue(vtk_face_id, gmds_face_id);
        m_gmds_to_vtk[2].emplace(gmds_face_id, vtk_face_id);

        m_vtk_grid->InsertNextCell(7,idList);
        vtk_face_id++;
    }

    m_vtk_BND = vtkIntArray::New();
    m_vtk_BND->SetName("BND_Color");
    m_vtk_BND->SetNumberOfValues(BND_faces.size());

    for(vtkIdType i = 0; i < m_vtk_grid->GetNumberOfCells(); i++){
        if(BND_Color->value(m_vtk_to_gmds[2]->GetValue(i)) > m_nb_geom_surfs){
            m_nb_geom_surfs = BND_Color->value(m_vtk_to_gmds[2]->GetValue(i));
        }
        m_vtk_BND->InsertValue(i,BND_Color->value(m_vtk_to_gmds[2]->GetValue(i)));
    }

    m_vtk_grid->GetCellData()->AddArray(m_vtk_BND);
    m_vtk_grid->GetCellData()->AddArray(m_vtk_to_gmds[2]);
}

m_vtk_cut = vtkIntArray::New();
m_vtk_cut->SetName("cut");

```

```

    m_vtk_cut->SetNumberOfValues(nbCells);

    for(vtkIdType i = 0; i < m_vtk_grid->GetNumberOfCells(); i++){
        m_vtk_cut->InsertValue(i,1);
    }
}
/*-----*/
MedusaGrid::~MedusaGrid() {
    m_vtk_grid->Delete();
    m_vtk_to_gmids[0]->Delete();
    m_vtk_to_gmids[1]->Delete();
    m_vtk_to_gmids[2]->Delete();
    m_vtk_to_gmids[3]->Delete();
    m_vtk_BND->Delete();
    m_vtk_cut->Delete();
    m_vtk_blocks->Delete();
}
/*-----*/
vtkUnstructuredGrid* MedusaGrid::grid() {return m_vtk_grid;}
/*-----*/
gmids::TCellID MedusaGrid::
getGMDSCellID(const int ADim, const vtkIdType &AVTKId)
{
    return (m_vtk_to_gmids[ADim])->GetValue(AVTKId);
}
/*-----*/
gmids::Mesh* MedusaGrid::getMesh() {return &m_gmids_mesh;}
/*-----*/
vtkIdType MedusaGrid::
getVTKCellID(const int ADim, gmids::TCellID AGMDSId)
{
    return m_gmids_to_vtk[ADim][AGMDSId];
}
/*-----*/
std::vector<gmids::TCellID> MedusaGrid::getSingularGraph() {
    Variable<int>* sing;
    std::vector<gmids::TCellID> sing_graph;
    try {
        sing = m_gmids_mesh.getVariable<int, GMDSCellID>("sing_tet");
        for(auto r : m_gmids_mesh.regions()){
            if(sing->value(r) > 0) {
                sing_graph.push_back(r);
            }
        }
    } catch (GMDSEException& e){
        std::cout<<"ERROR: Singularity variable on mesh not defined."<<std::endl;
    }

    return sing_graph;
}
/*-----*/
void MedusaGrid::
updateDualData()
{
    gmids::Variable<int>* block = m_gmids_mesh.getVariable<int, GMDSCellID>("blocks");

    for(vtkIdType i = 0; i < m_vtk_grid->GetNumberOfCells(); i++){
        TCellID gmids_id = getGMDSCellID(3,i);
        int value = (*block)[gmids_id];
        m_vtk_blocks->SetValue(i,value);
    }

    std::cout<<"Size " <<m_vtk_blocks->GetSize()<<std::endl;
    std::cout<<"Max value " <<m_vtk_blocks->GetDataTypeValueMax()<<std::endl;
    std::cout<<"NbValues " <<m_vtk_blocks->GetNumberOfValues()<<std::endl;
}
/*-----*/
void MedusaGrid::updateGrid(int test){

    m_vtk_grid->Reset();

    for(int i = 0; i<4; i++)
        m_gmids_to_vtk[i].clear();

    auto nbPoints = m_gmids_mesh.getNbNodes();
    auto nbCells = m_gmids_mesh.getNbRegions();

    //===== POINTS CREATION =====
    vtkPoints* grid_points = vtkPoints::New();
    grid_points->SetNumberOfPoints(nbPoints);
    grid_points->SetDataTypeToDouble();
    m_vtk_to_gmids[0]->SetNumberOfValues(nbPoints);

    std::map<TCellID, int> map_node_ids;
    int vtk_node_id=0;
    for(auto gmids_node_id:m_gmids_mesh.nodes()){
        //id mapping
        map_node_ids[gmids_node_id]=vtk_node_id;
        //point definition

```

```

    math::Point loc = m_gmgs_mesh.get<Node>(gmgs_node_id).getPoint();
    grid_points->InsertPoint(vtk_node_id, loc.X(), loc.Y(), loc.Z());
    m_vtk_to_gmgs[0]->InsertValue(vtk_node_id, gmgs_node_id);
    vtk_node_id++;
}
m_vtk_grid->SetPoints(grid_points);
grid_points->Delete();

m_vtk_grid->GetPointData()->AddArray(m_vtk_to_gmgs[0]);

if(test == 2){
    //===== FACES CREATION =====
    auto nbFaces = m_gmgs_mesh.getNbFaces();
    m_vtk_to_gmgs[2]->SetNumberOfValues(nbFaces);

    int vtk_face_id = 0;
    for (auto gmgs_face_id:m_gmgs_mesh.faces()) {
        Face f = m_gmgs_mesh.get<Face>(gmgs_face_id);
        std::vector<TCellID> node_ids = f.getIds<Node>();

        vtkIdList *idList = vtkIdList::New();
        idList->SetNumberOfIds(node_ids.size());
        int i_vtk_node = 0;
        for (auto i_node:node_ids) {
            idList->SetId(i_vtk_node++, map_node_ids[i_node]);
        }
        m_vtk_to_gmgs[2]->InsertValue(vtk_face_id, gmgs_face_id);
        m_gmgs_to_vtk[2].emplace(gmgs_face_id, vtk_face_id);

        m_vtk_grid->InsertNextCell(7, idList);
        vtk_face_id++;
    }
    m_vtk_grid->GetCellData()->AddArray(m_vtk_to_gmgs[2]);

    //std::cout << "nb VTK faces " << m_vtk_to_gmgs[2]->GetNumberOfValues() << std::endl;
} else {
    //===== CELLS CREATION =====
    m_vtk_grid->Allocate(nbCells);

    m_vtk_to_gmgs[3]->SetNumberOfValues(nbCells);

    int vtk_cell_id = 0;
    for (auto gmgs_cell_id:m_gmgs_mesh.regions()) {
        Region r = m_gmgs_mesh.get<Region>(gmgs_cell_id);
        int vtk_type = 0;
        switch (r.type()) {
            case GMDS_HEX:
                vtk_type = VTK_HEXAHEDRON;
                break;
            case GMDS_TETRA:
                vtk_type = VTK_TETRA;
                break;
            case GMDS_PRISM3:
                vtk_type = VTK_WEDGE;
                break;
            case GMDS_PYRAMID:
                vtk_type = VTK_PYRAMID;
                break;
            default:
                throw GMDSException("Input GMDS mesh contains cell not handled by VTK");
        }

        std::vector<TCellID> node_ids = r.getIds<Node>();

        vtkIdList *idList = vtkIdList::New();
        idList->SetNumberOfIds(node_ids.size());
        int i_vtk_node = 0;
        for (auto i_node:node_ids) {
            idList->SetId(i_vtk_node++, map_node_ids[i_node]);
        }
        m_vtk_to_gmgs[3]->InsertValue(vtk_cell_id, gmgs_cell_id);
        m_gmgs_to_vtk[3].emplace(gmgs_cell_id, vtk_cell_id);

        m_vtk_grid->InsertNextCell(vtk_type, idList);

        vtk_cell_id++;
    }

    std::cout << "nb VTK cells " << m_vtk_to_gmgs[3]->GetNumberOfValues() << std::endl;

    m_vtk_grid->GetCellData()->AddArray(m_vtk_to_gmgs[3]);
}
if(test == 0) {
    m_vtk_blocks->SetNumberOfValues(m_gmgs_mesh.getNbRegions());
}
}
/*-----*/
void MedusaGrid::UpdateBRep() {
    m_vtk_grid->Reset();
}

```

```

for(int i = 0; i<4; i++)
    m_gmds_to_vtk[i].clear();

gmds::Variable<int>* BND_Color = m_gmds_mesh.getVariable<int, GMDS_FACE>("BND_SURFACE_COLOR");
gmds::Variable<int>* cut_f = m_gmds_mesh.getVariable<int, GMDS_FACE>("CUT_face");

std::set<TCellID> nodes_set;
std::vector<TCellID> BND_faces;

for(auto f : m_gmds_mesh.faces()){
    if(BND_Color->value(f) != 0 && cut_f->value(f) == 0){
        Face face = m_gmds_mesh.get<Face>(f);
        BND_faces.push_back(f);
        for(auto n : face.getIds<Node>()){
            nodes_set.insert(n);
        }
    }
}

auto nbPoints = nodes_set.size();
auto nbFaces = BND_faces.size();

//===== POINTS CREATION =====
vtkPoints* grid_points = vtkPoints::New();
grid_points->SetNumberOfPoints(nbPoints);
grid_points->SetDataTypeToDouble();
m_vtk_to_gmds[0]->SetNumberOfValues(nbPoints);

std::map<TCellID, int> map_node_ids;
int vtk_node_id=0;
for(auto gmds_node_id:nodes_set){
    //id mapping
    map_node_ids[gmds_node_id]=vtk_node_id;
    //point definition
    math::Point loc = m_gmds_mesh.get<Node>(gmds_node_id).getPoint();
    grid_points->InsertPoint(vtk_node_id,loc.X(),loc.Y(),loc.Z());
    m_vtk_to_gmds[0]->InsertValue(vtk_node_id,gmds_node_id);
    vtk_node_id++;
}
m_vtk_grid->SetPoints(grid_points);
grid_points->Delete();

m_vtk_grid->GetPointData()->AddArray(m_vtk_to_gmds[0]);

//===== FACES CREATION =====
m_vtk_to_gmds[2]->SetNumberOfValues(nbFaces);

int vtk_face_id = 0;
for (auto gmds_face_id:BND_faces) {
    Face f = m_gmds_mesh.get<Face>(gmds_face_id);
    std::vector<TCellID> node_ids = f.getIds<Node>();

    vtkIdList *idList = vtkIdList::New();
    idList->SetNumberOfIds(node_ids.size());
    int i_vtk_node = 0;
    for (auto i_node:node_ids) {
        idList->SetId(i_vtk_node++, map_node_ids[i_node]);
    }
    m_vtk_to_gmds[2]->InsertValue(vtk_face_id, gmds_face_id);
    m_gmds_to_vtk[2].emplace(gmds_face_id, vtk_face_id);

    m_vtk_grid->InsertNextCell(7,idList);
    vtk_face_id++;
}

m_vtk_BND = vtkIntArray::New();
m_vtk_BND->SetName("BND_Color");
m_vtk_BND->SetNumberOfValues(BND_faces.size());

for(vtkIdType i = 0; i < m_vtk_grid->GetNumberOfCells(); i++){
    m_vtk_BND->InsertValue(i,BND_Color->value(m_vtk_to_gmds[2]->GetValue(i)));
}

m_vtk_grid->GetCellData()->AddArray(m_vtk_BND);
m_vtk_grid->GetCellData()->AddArray(m_vtk_to_gmds[2]);
}
/*-----*/
int MedusaGrid::getNbGeomSurfs() {
    return m_nb_geom_surfs;
}
/*-----*/
std::map<int, std::vector<TCellID>> MedusaGrid::getSingLines() {
    Variable<int>* sing;
    std::map<int, std::vector<gmds::TCellID>> sing_Lines;
    int line_ID = 0;
    try {
        sing = m_gmds_mesh.getVariable<int, GMDS_REGION>("sing_tet");
        m_gmds_mesh.unmarkAll<Region>(tet_treated);
        for(auto r : m_gmds_mesh.regions()){

```

```

    if(sing->value(r) > 0 && !m_gmds_mesh.isMarked<Region>(r, tet_treated)) {
        std::vector<TCellID> sing_line;
        sing_line.push_back(r);
        m_gmds_mesh.mark<Region>(r, tet_treated);
        Region tet = m_gmds_mesh.get<Region>(r);
        std::vector<TCellID> next_tets;
        for(auto n : tet.get<Node>()){
            for(auto n_r : n.getIDs<Region>()){
                if(sing->value(n_r) > 0 && !m_gmds_mesh.isMarked<Region>(n_r, tet_treated)){
                    m_gmds_mesh.mark<Region>(n_r, tet_treated);
                    next_tets.push_back(n_r);
                }
            }
        }
        while(!next_tets.empty()){
            TCellID current = next_tets.back();
            next_tets.pop_back();
            sing_line.push_back(current);
            Region tet_ = m_gmds_mesh.get<Region>(current);
            for(auto n : tet_.get<Node>()){
                for(auto n_r : n.getIDs<Region>()){
                    if(sing->value(n_r) > 0 && !m_gmds_mesh.isMarked<Region>(n_r, tet_treated)){
                        m_gmds_mesh.mark<Region>(n_r, tet_treated);
                        next_tets.push_back(n_r);
                    }
                }
            }
        }
        sing_lines.emplace(line_ID, sing_line);
        line_ID++;
    }
}

m_gmds_mesh.unmarkAll<Region>(tet_treated);
} catch (GMDSException e){
    std::cout<<"ERROR: Singularity variable on mesh not defined."<<std::endl;
}

return sing_lines;
}

/*-----*/
std::vector<TCellID> MedusaGrid::getOffset(int ADepth, std::vector<TCellID> AIDs) {

    std::vector<TCellID> result = AIDs;
    std::vector<TCellID> base = AIDs; //On part de Ãa pour faire le offset
    std::vector<TCellID> offset;
    m_gmds_mesh.unmarkAll<Region>(tet_treated);
    for (int i = 0; i < ADepth; i++) {
        for (auto t : base) {
            m_gmds_mesh.mark<Region>(t, tet_treated);
            Region r = m_gmds_mesh.get<Region>(t);
            for (auto n : r.get<Node>()) {
                for (auto n_t : n.getIDs<Region>()) {
                    if (!m_gmds_mesh.isMarked<Region>(n_t, tet_treated)) {
                        offset.push_back(n_t);
                        m_gmds_mesh.mark<Region>(n_t, tet_treated);
                    }
                }
            }
        }
        base = offset; //A la prochaine itÃration on partira du offset crÃÃ
        for(auto t : offset){
            result.push_back(t);
        }
        offset.clear();
    }

    return result;
}

/*-----*/
bool MedusaGrid::checkOffsetBoundary(std::vector<TCellID> AIDs){
    for(auto id : AIDs){
        Region r = m_gmds_mesh.get<Region>(id);
        for(auto f : r.get<Face>()){
            if(f.get<Region>().size() != 1){
                int cpt_tet = 0;
                for(auto n : f.get<Node>()){
                    for(auto t : n.getIDs<Region>()){
                        if(m_gmds_mesh.isMarked<Region>(t, tet_treated)) {
                            cpt_tet++;
                            break; //On passe au noeud d'aprÃs
                        }
                    }
                }
            }
        }
    }
}

m_gmds_mesh.unmarkAll<Region>(tet_treated);
return true;

```

```
}
/*-----*/
std::vector<TCellID> MedusaGrid::getNodes(std::vector<TCellID> ATetIDs) {
    std::set<TCellID> nodes;
    for(auto t : ATetIDs){
        Region r = m_gmds_mesh.get<Region>(t);
        for(auto n : r.getID<Node>()){
            nodes.insert(n);
        }
    }
    std::vector<TCellID> nodes_vec;
    nodes_vec.reserve(nodes.size());
    for(auto n : nodes){
        nodes_vec.push_back(n);
    }

    return nodes_vec;
}
```

```
/*-----*/
#ifndef GMDs_MEDUSAGRID_H
#define GMDs_MEDUSAGRID_H
/*-----*/
#include <vtkUnstructuredGrid.h>
#include <gmds/ig/Mesh.h>
#include <map>
#include <vtkIntArray.h>
/*-----*/
namespace medusa{
    class MedusaGrid{
    public:
        MedusaGrid(const std::string& AFileName);
        MedusaGrid();
        MedusaGrid(const gmds::Mesh* AMesh, int AMode);

        virtual ~MedusaGrid();

        vtkUnstructuredGrid* grid();

        gmds::TCellID getGMDSCellID(const int ADim, const vtkIdType& AVTKId);
        vtkIdType getVTKCellID(const int ADim, gmds::TCellID AGMDSId);

        gmds::Mesh* getMesh();

        std::vector<gmds::TCellID> getSingularGraph();

        void updateDualData();
        void updateCutData();

        void updateGrid(int test);

        void UpdateBRep();

        int getNbGeomSurfs();

        std::map<int, std::vector<gmds::TCellID>> getSingLines();

        std::vector<gmds::TCellID> getOffset(int ADepth, std::vector<gmds::TCellID> AIDs);

        bool checkOffsetBoundary(std::vector<gmds::TCellID> AIDs);

        std::vector<gmds::TCellID> getNodes(std::vector<gmds::TCellID> ATetIDs);

    public:
        std::string name;

    private:
        gmds::Mesh m_gmds_mesh;
        vtkUnstructuredGrid* m_vtk_grid;
        //we store the access to the mapping from vtk to gmds i-dim cell ids
        vtkIdTypeArray* m_vtk_to_gmds[4];

        vtkSmartPointer<vtkIntArray> m_vtk_blocks;
        vtkIntArray* m_vtk_cut;
        vtkIntArray* m_vtk_BND;
        std::map<gmds::TCellID, vtkIdType> m_gmds_to_vtk[4];

        int tet_treated = m_gmds_mesh.newMark<gmds::Region>();

        int m_nb_geom_surfs = 0;

        int m_mode = 0;

    };
}
/*-----*/
#endif //GMDs_MEDUSAGRID_H
/*-----*/
```



```
/*-----*/
#ifndef GMD5_MEDUSA_H
#define GMD5_MEDUSA_H
/*-----*/
#include <medusa/view/GraphicView.h>
#include <medusa/view/TextView.h>
#include <medusa/control/MediatorControl.h>
/*-----*/
namespace medusa {
    class Medusa {
    public:
        enum Mode{DUAL_BLOCKING, EXPLORATION, GEOMETRY};
        Medusa(const Mode AMode);
        void setMode(const Mode AMode);

        virtual ~Medusa();
        void load(const std::string& AFileName);
        void launch();
        GraphicView* newGraphicView(const std::string& AName, GraphicView::ViewType AType);
        TextView* newTextView();
    private:
        void init(const Medusa::Mode AMode);
    private:
        GraphicView* m_last_gview;
        std::vector<GraphicView*> m_gviews;
        std::vector<TextView*> m_tviews;

    };
}
/*-----*/
#endif //GMD5_MEDUSA_H
/*-----*/
```

```
/*-----*/  
#include <medusa/view/MedusaObserver.h>  
#include <medusa/model/MedusaBackEnd.h>  
/*-----*/  
using namespace medusa;  
using namespace gmds;  
/*-----*/
```

```
/*-----*/
#ifdef GMDS_MEDUSAOBSERVER_H
#define GMDS_MEDUSAOBSERVER_H
/*-----*/

namespace medusa{
    class MedusaObserver{
    public:
        virtual void update ()=0;

    };
}
/*-----*/
#endif //GMDS_MEDUSAOBSERVER_H
/*-----*/
```

```

/*-----*/
#include <gtest/gtest.h>

#include <medusa/view/GraphicView.h>
#include <medusa/view/TextView.h>
#include <medusa/Medusa.h>
/*-----*/
using namespace medusa;
/*-----*/
TEST(MedusaTest, init) {
    Medusa m(Medusa::EXPLORATION);
    m.newGraphicView("Medusa 1", GraphicView::View3D);
    m.newTextView();
}

```

```
/*-----*/
#include <medusa/control/MediatorControl.h>
#include "medusa/control/MouseInteractorStyle.h"
/*-----*/
using namespace medusa;
/*-----*/
vtkStandardNewMacro(MouseInteractorStyle);
/*-----*/
MouseInteractorStyle::MouseInteractorStyle() :
    vtkInteractorStyleTrackballCamera() {

}
/*-----*/
void MouseInteractorStyle::OnChar() {

    vtkInteractorStyleTrackballCamera::OnChar();
    switch (this->Interactor->GetKeyCode())
    {
        case 'P':
        case 'p':
            pick();
            break;

        case 'W':
        case 'w':
            wireframe();
            break;

        case 'O':
        case 'o':
            opacity();
            break;

        case 'H':
        case 'h':
            help();
            break;

        case 'G':
        case 'g':
            generate();
            break;

        case 'D':
        case 'd':
            viewDual();
            break;

        case 'S':
        case 's':
            singularityGraph();
            break;

        case 'R':
        case 'r':
            remove();
            break;

        case 'V':
        case 'v':
            surfaceMode();
            break;

        case 'U':
        case 'u':
            undo();
            break;

        case 'C':
        case 'c':
            color();
            break;

        case 'M':
        case 'm':
            mode();
            break;
    }
}
//-----
void MouseInteractorStyle::pick() {

    // Get the location of the click (in window coordinates)
    int *pos = this->GetInteractor()->GetEventPosition();

    vtkSmartPointer<vtkCellPicker> picker =
        vtkSmartPointer<vtkCellPicker>::New();
    picker->SetTolerance(0.0005);

    // Pick from this location.
    picker->Pick(pos[0], pos[1], 0, this->GetDefaultRenderer());
}
```

```
double *worldPosition = picker->GetPickPosition();

if (picker->GetCellId() != -1){
    MediatorControl::getInstance()->select(3, picker->GetCellId());
}
// Forward events
vtkInteractorStyleTrackballCamera::OnLeftButtonDown();
}
/*-----*/
void MouseInteractorStyle::setPolicy() {
    //MediatorControl::setPolicy();
}
/*-----*/
void MouseInteractorStyle::wireframe() {
    MediatorControl::getInstance()->wireframe();
}
/*-----*/
void MouseInteractorStyle::opacity() {
    MediatorControl::getInstance()->opacity();
}
/*-----*/
void MouseInteractorStyle::help() {
    MediatorControl::getInstance()->help();
}
/*-----*/
void MouseInteractorStyle::generate() {
    MediatorControl::getInstance()->generate();
}
/*-----*/
void MouseInteractorStyle::viewDual() {
    MediatorControl::getInstance()->toggleDual();
}
/*-----*/
void MouseInteractorStyle::remove() {
    MediatorControl::getInstance()->remove();

    int *pos = this->GetInteractor()->GetEventPosition();

    vtkSmartPointer<vtkCellPicker> picker =
        vtkSmartPointer<vtkCellPicker>::New();
    picker->SetTolerance(0.0005);

    // Pick from this location.
    picker->Pick(pos[0], pos[1], 0, this->GetDefaultRenderer());

    vtkInteractorStyleTrackballCamera::OnLeftButtonDown();
    if (picker->GetCellId() != -1){

        std::cout<<"Picker cell id "<<picker->GetCellId()<<std::endl;

        MediatorControl::getInstance()->removeItem(MediatorControl::getInstance()->removeActor(picker->GetActor()));
    }
}
/*-----*/
void MouseInteractorStyle::surfaceMode() {
    MediatorControl::getInstance()->surfaceMode();
}
/*-----*/
void MouseInteractorStyle::undo() {
    MediatorControl::getInstance()->undo();
}
/*-----*/
void MouseInteractorStyle::singularityGraph() {
    MediatorControl::getInstance()->singularityGraphToggle();
}
/*-----*/
void MouseInteractorStyle::color() {
    MediatorControl::getInstance()->color();
}
/*-----*/
void MouseInteractorStyle::mode() {
    MediatorControl::getInstance()->changePolicy();
}
```

```
/*-----*/
#ifndef GMDS_MOUSEINTERACTORSTYLE_H
#define GMDS_MOUSEINTERACTORSTYLE_H
/*-----*/

#include <vtkActor.h>
#include <vtkCellArray.h>
#include <vtkCellPicker.h>
#include <vtkCommand.h>
#include <vtkDataSetMapper.h>
#include <vtkExtractSelection.h>
#include <vtkIdTypeArray.h>
#include <vtkInteractorStyleTrackballCamera.h>
#include <vtkInteractorStyleSwitch.h>
#include <vtkNamedColors.h>
#include <vtkObjectFactory.h>
#include <vtkPlaneSource.h>
#include <vtkPoints.h>
#include <vtkPolyData.h>
#include <vtkPolyDataMapper.h>
#include <vtkProperty.h>
#include <vtkRenderWindow.h>
#include <vtkRenderWindowInteractor.h>
#include <vtkRenderer.h>
#include <vtkRendererCollection.h>
#include <vtkSelection.h>
#include <vtkSelectionNode.h>
#include <vtkSmartPointer.h>
#include <vtkTriangleFilter.h>
#include <vtkUnstructuredGrid.h>
/*-----*/

namespace medusa {
    class MouseInteractorStyle : public vtkInteractorStyleTrackballCamera {
    public:
        static MouseInteractorStyle *New();

        virtual void OnChar() override ;
        MouseInteractorStyle();
        void pick();
        void setPolicy();

        void wireframe();

        void solid();

        void opacity();

        void help();

        void generate();

        void viewDual();

        void remove();

        void surfaceMode();

        void finalMesh();

        void undo();

        void color();

        void singularityGraph();

        void mode();
    };
}
/*-----*/
#endif //GMDS_MOUSEINTERACTORSTYLE_H
/*-----*/
```

```

/*-----*/
#include <medusa/view/TextView.h>
#include <gmds/utils/Exception.h>
#include <iostream>
#include <medusa/model/MedusaBackend.h>
/*-----*/
using namespace medusa;
/*-----*/
TextView::TextView() {}
/*-----*/
void TextView::refresh() {
    throw gmds::GMDSException("Not yet implemented");
}
/*-----*/
void TextView::update() {
    std::cout<<"Update of text view"<<std::endl;
}
/*-----*/
void TextView::selectVTKCell(int ADIm, vtkIdType ACellID) {
    std::vector<MedusaGrid*> grids = MedusaBackend::getInstance()->grids();
    gmds::TCellID gmds_id = grids[0]->getGMDSCellID(ADIm,ACellID);
    std::cout<<"Selection ("<<ADIm<<","<<gmds_id<<")"<<std::endl;
}
/*-----*/
void TextView::createSurface(const std::vector<vtkIdType> AIDs, int ASheetID) {
    std::cout<<"Nb tet in dual surface "<<ASheetID<<": "<<AIDs.size()<<std::endl;
}
/*-----*/
void TextView::help() {
    std::cout<<"w - Wireframe"<<std::endl;
    std::cout<<"s - Solid"<<std::endl;
    std::cout<<"p - Pick/create surf"<<std::endl;
    std::cout<<"r - Remove"<<std::endl;
    std::cout<<"h - Help"<<std::endl;
    std::cout<<"g - Generate dual"<<std::endl;
    std::cout<<"b - Block generation"<<std::endl;
    std::cout<<"v - toggle surface View"<<std::endl;
}
/*-----*/
void TextView::generateDual() {
    std::cout<<"Dual generated"<<std::endl;
}
/*-----*/
void TextView::dualError() {
    std::cout<<"ERROR: The dual is not generated yet"<<std::endl;
}
/*-----*/
int TextView::getInputAxis() {
    char axis = '0';
    while(true){
        std::cout<<"Select dual surface normal direction : '1' for red, '2' for green or '3' to create a boundary surface"<<std::e
ndl;
        cin >> axis;
        switch(axis){
            case '1':
                return 1;
            case '2':
                return 2;
            case '3':
                return 3;
            case 'q':
                return 0;
            default:
                std::cout<<"Wrong input, please retry."<<std::endl;
                cin.clear();
        }
    }
}
/*-----*/
void TextView::remove(int ASheetID) {
    std::cout<<"The dual surface "<<ASheetID<<" has been removed"<<std::endl;
}
/*-----*/

```



```
/*-----*/
#ifndef GMDS_TEXTVIEW_H
#define GMDS_TEXTVIEW_H
/*-----*/
#include <medusa/view/View.h>
/*-----*/
namespace medusa{
    class TextView: public View {
    public:
        TextView();
        void selectVTKCell(int ADIm, vtkIdType ACellID);
        void refresh();
        void update();
        void remove(int ASheetID);

        void createSurface(const std::vector<vtkIdType> AIDs, int ASheetID);
        void help();
        void dualError();

        void generateDual();
        int getInputAxis();
    };
}
/*-----*/
#endif //GMDS_TEXTVIEW_H
/*-----*/
```

```
/*-----*/  
#include "medusa/view/View.h"  
/*-----*/
```

```
/*-----*/
#ifndef GMDS_VIEW_H
#define GMDS_VIEW_H
/*-----*/
#include <medusa/view/MedusaObserver.h>
/*-----*/
#include <vtkType.h>
#include <vector>
/*-----*/
namespace medusa {
    class View: public MedusaObserver {

    public:
        View() {}
        virtual ~View() {}
        virtual void refresh()=0;
        virtual void selectVTKCell(int ADIm, vtkIdType ACellID)=0;
        virtual void createSurface(const std::vector<vtkIdType> AIDs, int ASheetID)=0;
    };
}
/*-----*/
#endif //GMDS_VIEW_H
/*-----*/
```